

Raspberry Pi 4 Dual-Biometric Authentication System

Complete Technical Documentation — Explained for Computer Engineering Students

Hardware: Raspberry Pi 4 Model B (8 GB RAM), Raspberry Pi Camera Module v2, ILI9341 TFT Display (320×240, SPI), ESP32 + INMP441 I2S MEMS Microphone (audio source, UART link)

OS: Raspberry Pi OS Lite with XFCE desktop (64-bit, Debian Bookworm-based, Linux kernel 6.x)

How to read this document: This document assumes you understand operating system concepts — processes, threads, memory, interrupts, system calls, file descriptors, TCP/IP, and UART. It does **not** assume Python knowledge. Every Python-specific concept, library, and idiom is explained in full detail the first time it appears. Nothing is skipped.

Table of Contents

1. [Background: Python Runtime Concepts You Must Know First](#)
 2. [System Architecture Overview](#)
 3. [Face Recognition Subsystem](#)
 4. [Voice Biometric Subsystem](#)
 5. [Orchestra.py — The Orchestrator](#)
 6. [Client.py — The Remote TUI](#)
 7. [End-to-End Command Flow Examples](#)
 8. [Suggested Improvements for Commercial Grade](#)
-

1. Background: Python Runtime Concepts You Must Know First

Before diving into the system, you need to understand several Python-specific mechanisms that have no direct analogy in C or assembly. These are foundational to understanding everything else.

1.1 What Python Is at the Machine Level

Python is an **interpreted language**. When you run `python3 main.py`, the operating system launches the CPython interpreter — a C program (`/usr/bin/python3`). CPython reads your `.py` file, compiles it to an intermediate format called **bytecode** (stored in `.pyc` files), and then executes that bytecode instruction by instruction inside a virtual machine loop written in C.

This means Python code does not compile to native ARM machine instructions. Instead, every Python operation goes through the C interpreter. This is why Python is slower than C or Rust for computation-heavy tasks. The interpreter is the process — your Python script is data that the interpreter reads and executes.

Practical implication for this system: The neural network inference (the heaviest computation) is done outside the Python interpreter entirely — either in compiled Rust (for face) or in ONNX Runtime (for voice), both of which are native C/C++/Rust binaries that operate on raw memory without Python overhead.

1.2 The Global Interpreter Lock (GIL)

CPython has a mechanism called the **Global Interpreter Lock (GIL)**. This is a mutex — a mutual exclusion lock — built into the interpreter itself. The GIL rule is: **only one thread may execute Python bytecode at any given moment**, even on a multi-core CPU.

To understand why this exists: CPython manages memory using reference counting. Every Python object has a counter tracking how many variables point to it. When the counter reaches zero, the object is freed. On a multi-core system, two threads incrementing and decrementing the same counter simultaneously would cause data corruption (a classic race condition). Rather than put fine-grained locks on every object, the CPython developers chose one coarse lock — the GIL.

What the GIL means in practice:

- If you create 4 Python threads on a 4-core CPU, they cannot truly run in parallel. One thread holds the GIL and executes; the others wait. The GIL is released periodically (every 5 ms by default) so other threads get a turn.
- **Exception:** When a Python thread calls into C code (like NumPy's matrix multiply, or `socket.read()`), the C code can release the GIL while it runs, because C code does not touch Python objects. So NumPy math and socket I/O are genuinely parallel with the interpreter.
- **The workaround for full CPU parallelism:** Use `multiprocessing` — spawn separate OS processes. Each process is a completely independent CPython interpreter with its

own GIL. They truly run on separate CPU cores simultaneously. This is exactly what this system does.

1.3 What `pip` and `stdlib` Mean

`pip` stands for "Pip Installs Packages." It is Python's package manager — analogous to `apt` for Debian or `cargo` for Rust. When you run `pip install numpy`, `pip` downloads the `numpy` package from the Python Package Index (PyPI, at pypi.org) and installs it into the Python environment. The installed files go into a directory like `/usr/lib/python3/dist-packages/` or inside a virtual environment.

`stdlib` means "standard library." These are modules that ship with Python itself — no installation needed. Examples: `subprocess`, `json`, `os`, `time`, `socket`, `pickle`. They are written partly in Python and partly in C, and are always available.

1.4 What `asyncio` Is — Single-Threaded Concurrency

`asyncio` is Python's standard library module for writing concurrent code **without multiple threads**. This might sound paradoxical, so here is the explanation from first principles.

The problem `asyncio` solves: Imagine a WebSocket server that has 10 clients connected simultaneously. Using one thread per client would mean 10 OS threads — feasible, but threads have overhead (stack allocation, context switches, synchronization). The alternative is **event-driven I/O**, which is how Node.js, nginx, and many high-performance servers work.

How `asyncio` works at the OS level:

1. `asyncio` maintains an internal **event loop** — a single infinite loop running in a single thread.
2. All network sockets (WebSocket connections, TCP sockets) are registered with the OS using `epoll()` (on Linux) — a system call that lets one thread watch hundreds of file descriptors simultaneously for readiness.
3. The event loop calls `epoll_wait()` to block until any of the watched sockets have data to read or space to write.
4. When `epoll_wait()` returns, it tells the event loop which socket(s) became ready.
5. The event loop runs the corresponding handler for that socket, then goes back to `epoll_wait()`.

This is the same mechanism as `select()` / `poll()` / `epoll()` that you may have studied in OS courses, but wrapped in Python syntax.

The `async`/`await` syntax: In Python, functions that participate in this system are declared with `async def` instead of just `def`. They are called **coroutines**. When a coroutine needs to wait for I/O (like receiving a WebSocket message), it says `await` — which means "suspend this coroutine here, let the event loop run other coroutines, and resume me when the data is ready."

```
python

# Regular function - blocks the entire thread while waiting
def handle_client(websocket):
    message = websocket.recv()    # blocks everything for however long it takes

# Coroutine - suspends only itself, lets event loop do other things
async def handle_client(websocket):
    message = await websocket.recv()    # suspends HERE, event loop runs other c
```

From the OS's perspective: there is exactly **one thread** running in the asyncio event loop. The concurrency is cooperative — coroutines voluntarily yield control at `await` points. There is no pre-emption. This means there are no race conditions between coroutines (unlike threads), which is why the WebSocket message handling in Orchestra.py does not need mutex locks around most of its logic.

Why `asyncio` works for the orchestrator but not the workers:

- The orchestrator mostly waits for network events (WebSocket messages arriving, responses to send). These are all I/O waits, which `asyncio` handles perfectly.
- The face worker reads camera frames at 30 FPS in a tight loop, running MediaPipe on each frame. This is CPU-bound computation, not I/O waiting. `asyncio` cannot overlap CPU work — it can only overlap I/O waits. Putting the face worker in a coroutine would block the event loop for tens of milliseconds per frame, making the WebSocket server unresponsive. Hence: separate process.

1.5 What `numpy` Is

`numpy` (Numerical Python) is a Python library that provides **multi-dimensional arrays stored in contiguous C memory**. This is the key difference from a Python list.

A Python list like `[0.1, -0.2, 0.3]` stores three Python objects. Each Python object has at least 28 bytes of overhead (reference count, type pointer, etc.). The list itself stores pointers to these objects. Accessing element N requires two pointer dereferences and a bounds check.

A `numpy` array of 128 `float32` values is a single block of $128 \times 4 = 512$ bytes of contiguous memory, containing raw IEEE 754 single-precision floating point numbers. No Python objects, no overhead. This layout is identical to a C array `float arr[128]`.

When you call `numpy.dot(a, b)` (dot product of two vectors), numpy calls optimized C or BLAS/LAPACK routines that operate directly on these raw memory blocks using SIMD instructions (ARM NEON on the RPi4). Python is not involved in the inner loop at all.

Why this matters: The 128-D face embeddings and the Mel spectrogram tensors are numpy arrays. All math on them runs at C speed, not Python interpreter speed.

numpy data types: `dtype=np.float32` means each element is a 4-byte IEEE 754 single-precision float. `dtype=np.int16` means each element is a 2-byte signed integer.

1.6 What `pickle` Is

`pickle` is Python's built-in serialization module. It converts Python objects (dictionaries, lists, numpy arrays, class instances — anything) into a stream of bytes that can be written to a file and later read back, reconstructing the original object exactly.

Internally, pickle writes a sequence of opcodes (a mini binary format defined by the Python project). It is roughly analogous to how protobuf or msgpack serializes structured data, but specifically designed for Python objects.

The face database `faces_db.pkl` is a file containing a pickled Python dictionary. The keys are strings (person names). The values are numpy arrays (128-element float32 embeddings). When `load_database()` runs, it calls `pickle.load(file)`, which reads the bytes, interprets the opcodes, and reconstructs the dictionary in memory with all the numpy arrays restored.

Security note: Never unpickle data from an untrusted source. The pickle format can encode arbitrary Python operations including code execution. Since this database is stored locally on the RPi and only written by the system itself, this is not a concern here — but it is why "Suggested Improvements" section mentions encrypting the database.

1.7 What `JSON` Is in This Context

JSON (JavaScript Object Notation) is a text format for representing structured data. In Python, `json.dumps({"command": "reboot"})` converts the dictionary to the string `'{"command": "reboot"}'`. `json.loads('{"granted": true}')` converts it back to a Python dictionary `{"granted": True}`.

JSON is used for all WebSocket communication because:

- It is human-readable (useful for debugging)
- It works across different programming languages (Python on the RPi, Python on the laptop, potentially JavaScript in a web dashboard)
- It handles nested structures naturally
- Every WebSocket library on every platform can encode/decode it

Why not pickle for WebSocket messages? Pickle would also work, but it is Python-specific and produces binary output that is harder to debug. JSON is the standard for network APIs.

1.8 What a Python Dictionary Is

A Python dictionary (type `dict`) is a hash map — exactly like `std::unordered_map` in C++. Keys are hashed to find bucket positions. It stores key-value pairs. Access, insertion, and deletion are O(1) average. The command messages sent between processes and over WebSocket are Python dictionaries, serialized to JSON strings for network transport and pickle bytes for inter-process queues.

1.9 What `lambda` Is

In Python, `lambda` creates a small anonymous function inline. For example:

```
python
lambda: _sp.Popen(["sudo", "reboot"])
```

This creates a zero-argument function that, when called, runs `_sp.Popen(["sudo", "reboot"])`. It is equivalent to:

```
python
def unnamed_function():
    _sp.Popen(["sudo", "reboot"])
```

It is used in `call_later(2.0, lambda: ...)` because `call_later` needs a callable (function) to call after the delay — not the result of calling it. Passing `lambda: ...` passes the function itself; without `lambda`, the expression would execute immediately.

2. System Architecture Overview

The system runs as **three OS-level processes**, one per physical CPU core, coordinated through multiprocessing queues. A fourth process (the TUI client) runs on a separate Windows/Mac/Linux laptop and communicates over the local network.



2.1 Why Separate Processes (Not Threads)

As explained in Section 1.2, Python's GIL prevents true CPU parallelism between threads. The face worker does heavy OpenCV + numpy computation (MediaPipe detection, affine warp, matrix multiply for cosine similarity). The voice worker does ONNX inference + numpy FFT. If both ran as threads inside a single Python interpreter, they would take turns on a single core — even though the RPi4 has 4 cores available.

By using `multiprocessing.Process`, each worker is a completely separate CPython interpreter with its own GIL, its own heap, its own page table. The Linux scheduler assigns them to different physical cores. Each process runs on its core without GIL contention from the others.

CPU affinity pinning: The `os.sched_setaffinity()` system call tells the Linux scheduler "this process may only run on core N." Without this, the scheduler is free to migrate processes between cores (which it does to balance load). Migration causes **cache misses** — the L1/L2 cache on core 1 contains the face model weights, and moving the process to core 2 means those cache lines are cold. Pinning each worker to a dedicated core ensures the model weights and frequently-accessed arrays stay warm in that core's cache.

2.2 Why Not `asyncio` for Everything

The face worker's main loop calls `camera.read_preview()` at approximately 30 FPS. Each call blocks for up to 33 ms (one frame period) while the camera sensor captures and the DMA controller transfers the frame to RAM. Between frames, it runs MediaPipe on the frame, which takes another 15–30 ms of CPU computation. These are not I/O waits in the `epoll` sense — they are either DMA-blocking hardware reads or pure CPU computation. `asyncio` cannot make them concurrent with other work.

Similarly, the voice worker's `serial.Serial.read()` blocks the calling thread for up to 3 seconds (the serial timeout). If this were inside the `asyncio` event loop, the event loop would be completely frozen for up to 3 seconds while waiting for the UART — the WebSocket server would stop responding to the client entirely.

The solution: keep the orchestrator (network I/O, inherently async) in `asyncio`, and put the compute-heavy and blocking-I/O workers in separate OS processes where blocking is harmless.

2.3 Core Pinning Constants

```
CORE_SERVER = 0    # orchestrator (asyncio event loop + WebSocket)
CORE_FACE   = 1    # face worker (camera + inference)
```

```
CORE_VOICE = 2 # voice worker (UART + ONNX)
# Core 3 is left free for OS kernel tasks, interrupt handlers, DMA completion
```

The call `_pin(CORE_SERVER)` in `main()` calls `os.sched_setaffinity(0, {CORE_SERVER})`, where `0` means "current process" and `{CORE_SERVER}` is a Python set containing the allowed core numbers. This is a direct wrapper around the Linux `sched_setaffinity()` system call.

3. Face Recognition Subsystem

3.1 Files Involved

File	Role
<code>main.py</code>	Camera control, face detection, face alignment, enrollment/verification logic, TFT display driver, threshold constants
<code>rust_wrapper.py</code>	Python-to-Rust bridge — spawns the compiled Rust binary as a child process and parses its stdout output
<code>rust_face_engine/rust_face_engine</code>	Compiled Rust binary (ELF executable for ARM64) — loads MobileFaceNet via the <code>tract</code> crate, runs the neural network forward pass, outputs a 128-D embedding
<code>faces_db.pkl</code>	Pickle-serialized Python dictionary mapping name strings to 128-element float32 numpy arrays
<code>temp_enroll.jpg</code>	Temporary file: aligned 112×112 face crop written by Python, read by the Rust binary

3.2 Libraries Used — Detailed Explanations

`picamera2`

What it is: `picamera2` is the official Python interface to the Raspberry Pi camera hardware. It is installed via `apt install python3-picamera2` rather than pip, because it is tightly coupled to the RPi kernel and `libcamera` framework.

What `libcamera` is: `libcamera` is a Linux kernel-userspace camera framework. When the camera module is connected to the RPi's CSI-2 (Camera Serial Interface 2) port, the kernel's V4L2 (Video for Linux 2) driver exposes it as a device. `libcamera` sits between V4L2 and user code, handling sensor configuration (gain, exposure, white balance) and the ISP (Image Signal Processor) pipeline that converts raw Bayer sensor data into RGB frames.

How the camera DMA works: The camera sensor outputs Bayer-encoded data (raw pixel data where each pixel captures only one of R, G, or B). This data streams over the CSI-2 serial bus to the RPi's SoC. The RPi's camera ISP processes it (demosaicing, noise reduction, color correction) and writes the final RGB frame into a buffer in DRAM via DMA — without CPU involvement. `picamera2` gives Python code access to this DMA buffer.

Two modes used in this system:

- **Preview mode** (320×240 at ~30 FPS): The camera continuously captures frames. Each call to `camera.read_preview()` returns a numpy array of shape `(240, 320, 3)` — 240 rows, 320 columns, 3 color channels (RGB). The total size is $240 \times 320 \times 3 = \mathbf{230,400}$ bytes per frame.
- **Still mode** (3280×2464 for full-resolution capture): The camera's full sensor resolution. One frame is $3280 \times 2464 \times 3 = \mathbf{24,253,440}$ bytes ≈ 23 MB. This is only captured once per enrollment/verification, and the larger image gives MediaPipe more pixels to work with for precise keypoint localization.

`mediapipe`

What it is: MediaPipe is Google's open-source framework for on-device ML pipelines. This system uses one component: `mp.solutions.face_detection.FaceDetection`. This is a pre-trained neural network (BlazeFace architecture) that takes an RGB image as input and outputs:

- **Bounding boxes:** The rectangular region in the image containing each face (normalized coordinates 0.0 to 1.0)
- **6 keypoints per face:** (right eye, left eye, nose tip, mouth center, right ear trigion, left ear trigion) — also normalized coordinates

The `model_selection=0` parameter selects the short-range model, optimized for faces that are 0.5–2 meters from the camera. The `min_detection_confidence=0.5` parameter means a face is reported only if the model's confidence is at least 50%.

Why MediaPipe for detection instead of Rust? Face detection (finding where faces are in a frame) and face recognition (identifying who the face belongs to) are two different tasks. MediaPipe's BlazeFace is very fast (< 10 ms on RPi4 at 320×240), runs on every preview

frame, and provides keypoints needed for alignment. The Rust binary only handles the heavier identity-embedding step on the aligned hi-res crop.

opencv-python (cv2)

What it is: OpenCV (Open Source Computer Vision Library) is a C++ library with Python bindings. It is the industry standard for image processing. The Python `cv2` module is a thin wrapper — when you call `cv2.cvtColor(image, cv2.COLOR_RGB2BGR)`, Python passes the numpy array pointer to the underlying C++ function, which operates on the raw memory and returns a new numpy array.

Used for in this system:

- **Color conversion:** `cv2.cvtColor(img, cv2.COLOR_RGB2BGR)` — MediaPipe expects RGB (red-green-blue), OpenCV uses BGR (blue-green-red) internally, and the TFT display needs RGB. These conversions just reorder bytes in the array.
 - **Affine warp** (`cv2.warpAffine`): Applies a geometric transformation to align the face. This multiplies every pixel coordinate by a 2×3 matrix to produce new coordinates, then interpolates pixel values at the new positions. This is a standard linear algebra operation but optimized in C++ to run in milliseconds.
 - **Drawing** (`cv2.rectangle`, `cv2.putText`): Draws bounding boxes and text on the preview frame for the TFT display.
 - `cv2.imwrite`: Compresses a numpy array image into JPEG format and writes it to a file. The Rust binary reads this file.
-

numpy

As explained in Section 1.5, numpy provides C-backed contiguous arrays. In the face subsystem specifically:

- `np.linalg.norm(emb)` computes the Euclidean (L2) norm: $\sqrt{\sum_{i=0}^{127} x_i^2}$, in optimized C.
 - `emb / norm` divides every element of the array by the scalar — also in C, no Python loop.
 - `np.dot(a, b)` computes the dot product $\sum_{i=0}^{127} a_i \cdot b_i$ — this equals cosine similarity when both vectors are L2-normalized (unit length), because $\cos \theta = \frac{a \cdot b}{|a||b|}$ and if $|a| = |b| = 1$, then $\cos \theta = a \cdot b$.
-

`Pillow` (`PIL`)

What it is: Pillow is Python's image library. It handles image file format encoding/decoding (JPEG, PNG, BMP, etc.) and provides a `PIL.Image` object type.

Why it is needed alongside OpenCV: The Adafruit TFT display driver expects images as `PIL.Image` objects, not numpy arrays. The conversion is: numpy array (from OpenCV) → `PIL.Image.fromarray(array)` → Adafruit driver → SPI bytes → TFT display. This conversion is a thin wrapper — it does not copy the pixel data, just wraps the numpy array in a PIL object.

Also used for: Loading TrueType font files (`.ttf`) for drawing text on the TFT with `ImageFont.truetype()`. OpenCV's `cv2.putText` uses a simple bitmap font; Pillow's TrueType fonts look more professional.

`adafruit-circuitpython-rgb-display` and `adafruit-blinka`

What they are: Adafruit is a hardware maker that sells the ILI9341 TFT display. They provide Python drivers for their hardware. `adafruit-blinka` is a compatibility layer that lets Adafruit's CircuitPython (a MicroPython variant for microcontrollers) libraries run on a full Linux system like the RPi.

What `board` and `digitalio` provide: The `board` module maps Adafruit's pin names to RPi GPIO numbers. `digitalio.DigitalInOut(board.CE0)` creates a Python object representing the chip-enable GPIO pin, with methods `switch_to_output()`, `.value = True/False` to set high/low.

How the TFT display is driven:

The ILI9341 controller communicates over SPI (Serial Peripheral Interface). The RPi4 has a hardware SPI controller (SPI0) accessible as `/dev/spidev0.0`. The `busio.SPI()` object wraps this device. The Adafruit driver sends the pixel data at 64,000,000 Hz (64 MHz) SPI clock — that is 64 million clock cycles per second, 8 million bytes per second. One full frame (320×240×2 bytes = 153,600 bytes in RGB565 encoding) takes about 19 ms to transmit over SPI, which is why the TFT updates at a maximum of ~52 FPS (though the camera preview limits it to ~30 FPS).

`pickle` (`stdlib`)

As explained in Section 1.6, pickle serializes Python objects to bytes. The face database `faces_db.pkl` contains:

```
python
```

```
{  
    "alice": numpy.array([0.123, -0.456, ..., 0.789], dtype=float32), # 128 fl  
    "bob":  numpy.array([...], dtype=float32),  
    ...  
}
```

`load_database()` calls `pickle.load(open("faces_db.pkl", "rb"))`. `save_database(db)` calls `pickle.dump(db, open("faces_db.pkl", "wb"))`. `"rb"` and `"wb"` mean "read binary" and "write binary" — no text encoding/decoding.

`subprocess` (stdlib)

What it is: The `subprocess` module provides Python functions for spawning child processes. It is a Python wrapper around the POSIX `fork()`/`exec()`/`waitpid()` system calls.

`subprocess.run([RUST_BINARY_PATH, image_path], capture_output=True, text=True, check=True)` does:

1. `fork()` — creates a child process (copy-on-write clone of the Python interpreter)
2. In the child: `execve(RUST_BINARY_PATH, [RUST_BINARY_PATH, image_path], env)` — replaces the child's memory image with the Rust binary
3. Before `exec`: the child sets up pipes for stdout and stderr (because `capture_output=True`)
4. The parent calls `waitpid()` — blocks until the child exits
5. The parent reads the child's stdout and stderr through the pipes
6. If the child exits with a non-zero code and `check=True`, `subprocess.run()` raises a `CalledProcessError` exception

`capture_output=True` is shorthand for `stdout=subprocess.PIPE, stderr=subprocess.PIPE`. The OS creates two anonymous pipes — one for stdout, one for stderr. The child's stdout file descriptor is the write end; the parent reads from the read end.

`text=True` means the bytes read from stdout are decoded using UTF-8 to produce a Python string. Without this, `result.stdout` would be a `bytes` object (a Python bytes array).

`json` (stdlib)

`json.loads("[0.123, -0.456, ...]")` parses a JSON string and returns a Python list of floats.

The Rust binary prints its 128-D embedding using Rust's `{:?}` (debug format) for a `Vec<f32>`. This format produces `[0.123456, -0.456789, ...]` — which is valid JSON because JSON arrays use the same bracket-and-comma syntax. Python's `json.loads()` then converts this to a Python list, which is immediately passed to `np.array(vec, dtype=np.float32)` to create a numpy array.

3.3 The Rust Binary — Architecture and Data Flow

Why Rust Instead of Python for Inference

MobileFaceNet inference involves approximately 50 convolutional layers, batch normalization, and depthwise separable convolutions, all operating on a $112 \times 112 \times 3$ input tensor ($112 \times 112 \times 3 = 37,632$ float32 values = 150,528 bytes). Each layer multiplies, accumulates, and applies nonlinear functions across millions of multiply-accumulate (MAC) operations.

The Rust binary uses the `tract` crate — a pure-Rust ONNX inference engine. `tract` reads the ONNX model file at startup, then compiles the computation graph into a sequence of optimized ARM64 instructions (using ARM NEON SIMD intrinsics for vectorized math). Each MobileFaceNet inference takes approximately 50–150 ms on the RPi4.

In comparison, Python + ONNX Runtime on ARM takes 200–400 ms for the same model. This is because Python ONNX Runtime is a C++ library but has Python overhead for data marshaling, whereas `tract` operates entirely in Rust with zero Python involvement.

The additional startup cost: Every time `rust_wrapper.py` calls `subprocess.run([RUST_BINARY_PATH, image_path])`, a new OS process is created and the Rust binary starts fresh — loading and preparing the ONNX model takes approximately 200–500 ms every time. This means each face embedding call costs: process creation (~5 ms) + model loading (~200–500 ms) + inference (50–150 ms) + IPC overhead (~1 ms) $\approx$ **250–650 ms total**. The "Suggested Improvements" section addresses this.

How Python Calls the Rust Binary — Step by Step

```

Python main.py
├─ cv2.imwrite("temp_enroll.jpg", aligned_face_crop)  ← writes 112×112
JPEG to disk
├─ rust_wrapper.get_face_embedding("temp_enroll.jpg")
│   └─ subprocess.run(["./rust_face_engine/rust_face_engine",
"temp_enroll.jpg"])
│       │
│       ├─ fork()                                ← OS creates child process
│       │
│       ├─ [child] execve(...)                    ← child becomes Rust binary
│       │
│       ├─ [Rust] open("temp_enroll.jpg")
│       ├─ [Rust] decode JPEG
│       ├─ [Rust] resize to 112×112
│       ├─ [Rust] normalize pixels
│       ├─ [Rust] MobileFaceNet forward pass (tract)
│       ├─ [Rust] L2 normalize output
│       ├─ [Rust] print "[0.123, -0.456, ...]" to stdout
│       └─ [Rust] exit(0)
│
├─ [parent] waitpid()                            ← parent waits for child to
finish
├─ [parent] read stdout pipe                      ← parent reads Rust's
output
│   │
│   └─ json.loads(stdout)                        ← parse JSON array
│       │
│       └─ np.array(..., dtype=float32)          ← make
numpy array
│           │
│           └─ defensive L2 normalize            ← in
Python
│               │
│               └─ return 128-D numpy
array

```

What the Rust Binary Receives and Outputs

- **Input:** One command-line argument — the file path to a JPEG image (the aligned face crop)

- **Output to stdout:** A single line: `[f0, f1, f2, ..., f127]` — 128 comma-separated float32 values in square brackets. Example: `[0.08372405, -0.11234567, 0.03456789, ...]`
- **Output to stderr:** Diagnostic messages (model loading status, errors) — captured by Python but only displayed if an error occurs
- **Exit code:** 0 for success, non-zero for failure (file not found, inference error, etc.)

Defensive Re-Normalization in Python

After parsing the Rust output, `rust_wrapper.py` runs:

```
python
norm = float(np.linalg.norm(emb))
if norm > 1e-10:
    emb = emb / norm
```

The Rust binary already applies L2 normalization. Python does it again as a safety measure — in case of numerical drift during serialization (printing float32 values as decimal strings and reparsing them introduces tiny rounding errors), or in case of a future Rust binary version that does not normalize. This costs < 1 ms and ensures the embedding always has unit length before entering the comparison functions.

3.4 Face Alignment — Why It Matters

The MobileFaceNet model was trained on a dataset where every face was aligned to a canonical pose: eyes at 35% from the top of a 112×112 crop, with inter-eye distance equal to 37% of the width. If you present a rotated or off-center face to the model, the pixels that represent "left eye" are in the wrong position in the input tensor. The model has no built-in rotation invariance. The embedding for a tilted face will differ significantly from the embedding for the same person's upright face.

Face alignment eliminates this variance by computing a 2D affine transformation (rotation + scale + translation) that moves the detected eyes to the canonical target positions, then applying that transformation to warp the face crop into the standard 112×112 layout before sending it to the Rust binary.

Mathematical steps in `align_face()`:

1. MediaPipe gives eye keypoints as (x, y) pairs in normalized coordinates (0.0 to 1.0). Multiply by image dimensions to get pixel coordinates.
2. Compute head rotation angle:

```
angle = arctan2(left_eye_y - right_eye_y, left_eye_x - right_eye_x)
```

This is the angle in radians that the eye-to-eye line makes with the horizontal. If the head is tilted 15° clockwise, `angle = -15° = -0.2618 rad`.

3. Compute scale factor:

```
actual_eye_dist = sqrt((lx-rx)² + (ly-ry)²) [pixels in hi-res image]
desired_eye_dist = 0.37 × 112 = 41.44 pixels [in output crop]
scale = desired_eye_dist / actual_eye_dist
```

4. `cv2.getRotationMatrix2D(center, angle_degrees, scale)` returns a 2×3 affine matrix:

```
M = [ α   β   (1-α)cx - β·cy ]
     [-β   α   β·cx + (1-α)cy ]
where α = scale·cos(angle), β = scale·sin(angle), (cx,cy) = rotation
center
```

5. Adjust the translation column of M so the midpoint between the eyes lands at the target position in the 112×112 output.

6. `cv2.warpAffine(image, M, (112, 112))` applies the transformation: for each output pixel (x', y'), compute source coordinates $(x, y) = M^{-1} \cdot (x', y')$, then bilinearly interpolate the source image at (x, y) to get the output pixel value.

The result is always a 112×112 RGB image where the face is upright, centered, and at the correct scale — regardless of how the person was standing relative to the camera.

3.5 Camera Dual-Mode Operation

The PiCamera2 library cannot run preview and still capture simultaneously. The two capture modes have different sensor configurations (binning, readout speed, noise characteristics), and `libcamera` requires a full stop-reconfigure-restart cycle to switch between them.

Preview mode (320×240 RGB888, ~30 FPS):

- Full sensor (8 MP) with 4×4 pixel binning → 320×240 output
- Low latency, continuous stream
- Used for: face detection on every frame, TFT display, bounding box tracking

Still mode (3280×2464 RGB888):

- Full sensor readout, no binning → 8 MP output
- Higher quality, more pixels for keypoint detection
- Used only when enrollment or verification is triggered

Transition sequence:

1. `camera._to_hires()`: stops the camera (sends stop command to `libcamera`), creates a new camera configuration for still capture, restarts. Takes ~800 ms because the sensor must reconfigure its timing.
2. Capture one hi-res frame (`camera.capture_array()`)
3. Run face detection and alignment on hi-res frame
4. Write aligned 112×112 crop to `temp_enroll.jpg`
5. `camera._to_preview()`: stops, reconfigures for preview, restarts. Takes ~500 ms.
6. TFT resumes updating

During these ~1.3 seconds, the TFT display is frozen (last frame remains visible). This is acceptable for the enrollment/verification use case but would be undesirable in a production system.

3.6 Enrollment and Verification Flow

Enrollment — complete sequence:

1. User types a name on the client laptop and clicks "Enroll Face"
2. Client sends JSON over WebSocket: `{"command": "register_face", "name": "Alice"}`
3. Orchestra receives the message, calls `_route()`, dispatches to face worker:
`face_cmd_q.put({"op": "enroll", "name": "Alice"})`
4. Face worker's main loop checks the command queue, picks up the command, calls
`_handle_command({"op": "enroll", "name": "Alice"})`
5. Countdown loop:
 - TFT displays "LOOK AT CAMERA - 3s" (green text on black)
 - Every 100 ms: capture preview frame → run MediaPipe → check face conditions
 - If face is too small (< 60 px wide in preview ≈ person too far away) → TFT shows "MOVE CLOSER!" → reset countdown
 - If multiple faces detected → TFT shows "ONE FACE ONLY" → reset countdown
 - If no face detected → TFT shows "NO FACE" → reset countdown
 - If conditions pass for 3 consecutive seconds → countdown completes

6. Switch to hi-res mode
7. Capture 3280×2464 frame
8. Run MediaPipe on hi-res frame → get eye keypoints
9. `align_face()` → 112×112 aligned crop
10. `cv2.imwrite("temp_enroll.jpg", aligned_crop_bgr)` → write to disk
11. `get_face_embedding("temp_enroll.jpg")` → spawn Rust binary → get 128-D embedding
12. **Duplicate check:** for every name in `faces_db`:
 - `score = np.dot(new_emb, existing_emb)` (both unit-length, so this = cosine similarity)
 - If `score >= 0.80` (`ENROLL_DUPLICATE_THRESHOLD`): reject enrollment, send error: "Face matches existing user 'Bob' (score 0.83)"
13. If unique: `db["Alice"] = new_embedding`, `pickle.dump(db, ...)` → save to `faces_db.pkl`
14. Switch back to preview mode
15. `face_res_q.put({"status": "success", "message": "Alice enrolled"})` → result travels back to Orchestra → JSON sent over WebSocket → displayed in client log

Autonomous verification loop (no command needed):

The face worker runs this continuously:

1. Capture preview frame
2. MediaPipe detection
3. If exactly one face, size OK, and at least 2 seconds since last verification attempt:
 - **Bounding box tracking:** if this face overlaps with the previously verified face by >30% (IoU metric), and the last result was GRANTED, skip re-verification — it is the same person still standing there
 - Otherwise: switch to hi-res → capture → align → embed via Rust
 - `match_probe()`: compare embedding against all enrolled templates
 - Three-layer rejection (see next section)
 - If GRANTED: TFT shows "✓ GRANTED - Alice" in green, log event to `shared_logs`
 - If DENIED: TFT shows "X UNKNOWN" in red, log event

3.7 The Three-Layer Rejection Logic

This is implemented in the `match_probe()` function in `main.py`. It uses three independent

checks, all of which must pass for access to be granted.

Layer 1 — Gallery size guard ($\text{MIN_GALLERY_SIZE} = 2$)

With only 1 person enrolled, the comparison has no reference. A stranger's face produces a cosine similarity score against "Alice" — maybe 0.45. The threshold is 0.75, so the stranger is denied. But in poor lighting or an edge case, the stranger might score 0.78. With only one enrolled template, the system cannot detect this as anomalous.

With 2+ people enrolled, the gap check (Layer 3) provides safety: the stranger might score 0.78 vs Alice and 0.76 vs Bob. The gap is only 0.02, which is below $\text{MIN_SCORE_GAP} = 0.10$, so the match is ambiguous and denied.

Therefore: if fewer than 2 people are enrolled, verification returns DENIED immediately with the message "Not enough enrolled identities."

Layer 2 — Absolute threshold ($\text{SIMILARITY_THRESHOLD} = 0.75$)

After computing cosine similarity against all enrolled templates, find the best (highest) score. If it is below 0.75, the face does not match anyone closely enough — deny.

- Genuine same-person matches: typically 0.80–0.92
- Impostors: typically 0.30–0.55
- Edge cases (family members, look-alikes): 0.55–0.75 → correctly denied by this threshold

Layer 3 — Gap check ($\text{MIN_SCORE_GAP} = 0.10$)

Compute: $\text{best_score} - \text{second_best_score}$. If this gap < 0.10 , the identity is ambiguous — deny.

Example: Bob scores 0.82, Alice scores 0.79. The gap is 0.03. This means the system cannot confidently say "this is Bob and not Alice." Result: DENIED, even though Bob's score is above the absolute threshold.

This layer catches a specific attack: if someone resembles two enrolled users, they might score just above threshold against both, and the system would incorrectly grant access. The gap check prevents this.

4. Voice Biometric Subsystem

4.1 Files Involved

File	Role
<code>rpia_ecapa_voice_biometric_v2.py</code>	UART audio receiver, Mel spectrogram computation, ONNX inference, enrollment/verification logic, quality checks
<code>ecapa_tdnns.onnx</code>	Pre-trained ECAPA-TDNN speaker embedding model in ONNX format
<code>voiceprint_database_onnx/</code>	Directory containing one <code>.npy</code> file per enrolled speaker (e.g., <code>voiceprint_database_onnx/alice.npy</code>)

4.2 Libraries Used — Detailed Explanations

`onnxruntime`

What ONNX is: ONNX (Open Neural Network Exchange) is an open file format for representing neural network models. A `.onnx` file stores the model's computation graph: which operations to perform (convolution, batch norm, ReLU, etc.), their weights (the trained numbers), and the connections between them. It is framework-independent — a model trained in PyTorch can be exported to ONNX and run with ONNX Runtime, TensorFlow, ncnn, TVM, or `tract` (Rust).

What ONNX Runtime is: ONNX Runtime is Microsoft's inference engine for ONNX models. It is a C++ library with Python bindings. When you call:

```
python
session = onnxruntime.InferenceSession("ecapa_tdnns.onnx")
```

ONNX Runtime reads the `.onnx` file, analyzes the computation graph, and compiles it into an optimized execution plan. On the RPi4 (ARM64 without GPU), it uses optimized CPU kernels, possibly using ARM NEON SIMD for matrix operations.

To run inference:

```
python
```

```
output = session.run(None, {input_name: features_numpy_array})
```

This passes the numpy array directly to ONNX Runtime (zero copy — ONNX Runtime reads the numpy array's underlying C memory pointer) and returns the output as a list of numpy arrays.

The voice worker configures ONNX Runtime with:

```
python

opts = onnxruntime.SessionOptions()
opts.intra_op_num_threads = 2 # use 2 CPU threads for internal parallelism wi
```

This means ONNX Runtime can parallelize operations within one forward pass (e.g., computing multiple convolution output channels simultaneously), using up to 2 threads. These threads run on Core 2, so they do not interfere with Core 1 (face) or Core 0 (orchestrator).

`pyserial` (`serial`)

What it is: `pyserial` is a Python library that provides an object-oriented interface to serial (UART) ports on Linux, Windows, and macOS. On Linux, a serial port is exposed as a character device file (`/dev/serial0` on RPi4).

How `serial.Serial` works at the OS level:

```
python

ser = serial.Serial(
    port="/dev/serial0",
    baudrate=460800,
    bytesize=serial.EIGHTBITS, # 8 data bits per frame
    parity=serial.PARITY_NONE, # no parity bit
    stopbits=serial.STOPBITS_ONE, # 1 stop bit
    timeout=3.0) # read timeout in seconds
```

`serial.Serial.__init__()` internally calls `open("/dev/serial0", O_RDWR | O_NOCTTY)` to get a file descriptor, then calls `tcgetattr(fd, &termios)` and `tcsetattr(fd, ...)` to configure the serial port. `tcsetattr` is a POSIX system call that sets the tty (terminal) parameters in the kernel driver: baud rate, data bits, parity, stop bits, and timeout.

The `timeout=3.0` parameter sets `termios.c_cc[VTIME] = 30` (in tenths of a second) and `termios.c_cc[VMIN] = 0`, which tells the kernel: "a `read()` call on this fd should return after 3 seconds even if fewer than the requested bytes arrived."

`numpy` (voice-specific uses)

- `np.frombuffer(raw_pcm_bytes, dtype=np.int16)` — interprets the raw byte array as an array of 16-bit signed integers without any data copying. This is a zero-copy operation: numpy creates an array object whose data pointer points directly into the existing byte buffer.
 - `audio.astype(np.float32) / 32768.0` — converts int16 samples to float32 in range $[-1.0, 1.0]$. Division by 32768 (2^{15}) maps the maximum positive int16 value (32767) to approximately 1.0.
 - `np.hanning(WIN_LENGTH)` — generates a Hann window: $w[n] = 0.5 \cdot (1 - \cos(\frac{2\pi n}{N-1}))$, a smooth taper that reduces spectral leakage in the FFT.
 - `np.fft.rfft(windowed_frame, n=N_FFT)` — real-input FFT. For a real-valued input of N samples, the FFT output has $N/2+1$ complex values (the rest are conjugate symmetric and redundant). With `N_FFT=400`, output has 201 complex values.
 - The Mel filterbank is a (80, 201) float32 numpy matrix precomputed once at startup. Each row is one triangular Mel filter. `mel_energies = mel_filterbank @ power_spectrum` computes 80 Mel filter bank energies from 201 FFT magnitude bins in one matrix multiply.
 - `np.save("alice.npy", embedding)` saves the numpy array directly to disk in `.npy` format — a simple header followed by the raw float32 bytes. `np.load("alice.npy")` reads it back. This is faster than pickle for pure numpy arrays.
-

`psutil`

What it is: `psutil` (Process and System Utilities) is a Python library that provides CPU usage, memory usage, and other system metrics for a specific process or the whole system.

Used in `verify_speaker()` to measure how much CPU and RAM the voice processing consumed:

```
python

proc = psutil.Process(os.getpid()) # "my process"
cpu = proc.cpu_percent(interval=None)
ram = proc.memory_info().rss / (1024*1024) # resident set size in MB
```

4.3 UART Communication — Deep Technical Details

4.3.1 Physical Layer

The ESP32 microcontroller captures audio from an INMP441 I2S MEMS microphone:

- **INMP441 to ESP32:** The microphone connects via I2S (Inter-IC Sound) bus. I2S is a synchronous serial protocol specifically designed for audio data. The ESP32 generates the I2S clock and receives the microphone's digital audio samples.
- **ESP32 to RPi4:** After buffering a frame's worth of samples, the ESP32 transmits them over UART (Universal Asynchronous Receiver-Transmitter) to the RPi4's hardware UART on GPIO pin 15 (RXD0).

The UART connection parameters:

- **Port on RPi:** `/dev/serial0` — the PL011 UART (the high-quality UART on RPi4, as opposed to the mini-UART which has limitations at high baud rates)
- **Baud rate:** 460,800 bits per second
- **Frame format:** 8N1 — 8 data bits, No parity, 1 stop bit

4.3.2 UART Framing — What 8N1 Means

UART is asynchronous: there is no shared clock line. The transmitter and receiver must agree on the bit rate (baud rate) in advance. Each byte is framed with:

```
Idle (line HIGH)
  |
  ▼
Start bit (LOW, 1 bit period)
  |
  ▼
Data bits (8 bits, LSB first)
  |
  ▼
Stop bit (HIGH, 1 bit period)
  |
  ▼
Idle (line HIGH) or next start bit
```

One UART character transmission = 1 start + 8 data + 1 stop = **10 bit periods**.

At 460,800 baud (460,800 bit periods per second): one bit period = $1/460800 = \mathbf{2.17 \text{ microseconds}}$.

One byte transmission time = $10 \times 2.17 \mu\text{s} = \mathbf{21.7 \text{ microseconds}}$.

4.3.3 Baud Rate and Data Rate Calculation

```
Baud rate:      460,800 bits/second
Bit period:     1 / 460,800 = 2.17 μs

Bytes per second: 460,800 / 10 = 46,080 bytes/second

Audio parameters:
  Sample rate:   16,000 samples/second
  Sample width:  2 bytes (16-bit PCM)
  Raw data rate: 16,000 × 2 = 32,000 bytes/second

UART utilization: 32,000 / 46,080 = 69.4%
```

The 69.4% utilization means 30.6% of the UART bandwidth is "wasted" on start/stop bits and gaps between frames. This is acceptable — the UART is fast enough to carry the audio in real-time with comfortable headroom for the framing protocol overhead (sync bytes, length bytes).

Why 460,800 baud instead of a standard like 115,200?

115,200 baud → 11,520 bytes/second. The audio stream requires 32,000 bytes/second. 115,200 baud is only 36% of what is needed — the RPi could not keep up with real-time audio at this rate. 460,800 baud is a supported rate on both the ESP32 and the RPi4's PL011 UART, and provides the required throughput.

4.3.4 What One Audio Sample Means

A single audio sample is a **16-bit signed integer (int16)**, stored in **2 bytes** in little-endian format (low byte first).

- Range: -32,768 to +32,767
- Represents: instantaneous air pressure/displacement at a specific moment in time
- 0 = silence (no pressure change)
- $\pm 32,767$ = maximum loudness (full scale)

At a sample rate of 16,000 samples per second, the time between consecutive samples is:

```
1 / 16,000 = 62.5 microseconds
```

One second of audio:

```
16,000 samples × 2 bytes = 32,000 bytes
```

The Nyquist theorem states that a sample rate of 16,000 samples/second can accurately represent audio frequencies up to 8,000 Hz ($16,000 / 2$). Human speech energy is concentrated between 300 Hz and 3,400 Hz for telephony, or up to ~8,000 Hz for high-quality speech. 16,000 Hz sample rate is the standard for speech recognition and voice biometrics — it captures all relevant speech frequencies.

4.3.5 Frame Protocol (ESP32 → RPi4)

The ESP32 does not send raw bytes continuously. It wraps each burst of PCM samples in a **frame** with a header that allows the receiver to synchronize:

```
Byte 0:  0xAA          ← sync byte 1 (fixed value)
Byte 1:  0x55          ← sync byte 2 (fixed value)
Byte 2:  LEN_H          ← high byte of payload length
Byte 3:  LEN_L          ← low byte of payload length
Bytes 4...(4+LEN-1): PCM ← raw 16-bit PCM samples, little-endian

Special case: LEN_H=0, LEN_L=0 → End Of Session marker (no payload)
```

Why the sync header (0xAA 0x55)?

In a binary protocol over UART, there is no inherent way to know where a frame starts. If the receiver starts reading in the middle of a transmission (after power-on, or after a glitch), it will misalign and interpret data bytes as header bytes. The sync sequence solves this:

The receiver scans byte-by-byte until it sees 0xAA immediately followed by 0x55. Once found, it knows the next 2 bytes are the length. This is called **byte synchronization** or **framing**.

The sync values are chosen to be relatively rare in audio PCM data. The probability that two consecutive PCM bytes happen to be 0xAA followed by 0x55 is $1/(256 \times 256) = 1/65,536 \approx 0.0015\%$. In practice, false syncs occasionally occur (especially in loud audio), which is why the length field provides a second validation: if `LEN` is unreasonably large or the subsequent bytes do not form valid audio, the receiver knows it has a false sync.

The length field:

The 2-byte big-endian length `(LEN_H << 8) | LEN_L` gives the number of PCM data bytes in this frame (not the total frame size). Maximum length: 65,535 bytes = 32,767 samples $\approx$ 2.05 seconds of audio per frame. In practice, the ESP32 firmware sends frames of 512–2048 bytes at a time, corresponding to 256–1024 samples = 16–64 ms of audio per frame.

The `_sync()` state machine in Python:

```
python
```

```
def _sync(self):
    state = 0
    while True:
        b = self._ser.read(1)    # blocks until 1 byte arrives (or timeout)
        if not b: return False    # timeout → no sync found
        v = b[0]                 # extract the integer value from bytes object
        if state == 0:
            state = 1 if v == 0xAA else 0    # found first sync byte → state 1
        else:
            # state 1: waiting for 0x55
            if v == 0x55:
                return True    # found! sync complete
            state = 1 if v == 0xAA else 0    # 0xAA again: could be start of new
```

This is a two-state FSM (Finite State Machine):

- State 0 (WAITING): looking for 0xAA
- State 1 (SEEN_AA): saw 0xAA, now looking for 0x55
- If we are in state 1 and see 0xAA again: the previous 0xAA was not a sync start, but this one might be → stay in state 1
- If we are in state 1 and see anything other than 0x55 or 0xAA: reset to state 0

4.3.6 How the RPi Knows Data Is Arriving — The Full Hardware-to-Python Stack

This is the most important section for understanding the low-level operation. The data path from ESP32 to Python involves four layers:

Layer 1 — UART Hardware (PL011 controller in RPi4 SoC)

The RPi4's ARM Cortex-A72 SoC contains a PL011 UART peripheral (Arm's PrimeCell UART). This hardware:

- Monitors the RX pin continuously, sampling it at $16 \times$ the baud rate ($460,800 \times 16 = 7,372,800$ Hz) to detect start bits and sample data bits accurately
- When the start bit is detected, shifts in 8 data bits at the baud rate timing
- After receiving a complete byte, stores it in the RX FIFO (a 32-byte hardware FIFO built into the PL011 silicon)
- When the FIFO reaches the configured threshold (typically 1/2 full = 16 bytes), or after a character gap (no new bytes for ~ 3 bit periods), the PL011 raises an interrupt signal on the ARM GIC (Generic Interrupt Controller)

The ARM GIC triggers the interrupt vector table. The Linux kernel's PL011 driver ISR runs in interrupt context (highest priority, interrupts disabled):

1. The ISR reads bytes from the PL011 hardware FIFO by reading the data register at the UART's MMIO address
2. Copies them into the kernel's **tty flip buffer** (a software ring buffer in kernel space, default 4096 bytes, expandable)
3. Clears the interrupt flag in the PL011
4. Returns from interrupt context

This entire ISR runs in approximately 2–10 microseconds. No Python code is involved. The bytes are now in kernel RAM.

Layer 3 — tty subsystem → file descriptor → `read()` system call

When the Python voice worker calls `serial.Serial.read(n)`, pyserial calls the C standard library `read(fd, buf, n)` function, which issues the `read` system call (syscall number varies by ARM64 kernel version, typically 63).

The kernel checks the tty flip buffer:

- If $\geq n$ bytes are available: copy them to userspace (`buf`), return `n`
- If $< n$ bytes available and the read timeout has not expired: put the Python process to sleep (set process state to `TASK_INTERRUPTIBLE`, remove from run queue)
- The tty driver will wake the process when enough bytes arrive or the timeout expires

Layer 4 — Python user space

After `read()` returns, pyserial packages the returned bytes into a Python `bytes` object and returns it. The Python voice worker code then processes these bytes.

Summary flow:

ESP32 TX pin changes voltage

- PL011 RX pin samples it at 7.37 MHz
- PL011 assembles byte, stores in hardware FIFO
- FIFO threshold reached → PL011 raises interrupt
- ARM GIC signals CPU core 2 (or whichever core runs ISR)
- Linux kernel PL011 ISR fires (microseconds)
- ISR copies bytes to tty flip buffer in kernel RAM
- ISR returns, normal execution resumes
- Python process calls `read(fd, buf, n)`
- kernel copies bytes from flip buffer to userspace buf
- `read()` returns n bytes
- `pyserial` wraps bytes in Python bytes object
- `_read_frame()` processes the bytes

Clarification: Is this "interrupt-driven" from Python's perspective?

From Python's perspective, it is **blocking I/O with a timeout**. Python calls `read()`, the OS puts the Python process to sleep, and the process wakes up when bytes arrive. The interrupt-driven part is entirely inside the kernel — Python never sees interrupts. From the OS perspective, yes, the UART is interrupt-driven: the kernel uses interrupts to know when bytes arrive and to wake the sleeping Python process.

4.3.7 The Stale Buffer Glitch Fix

Between voice sessions, the kernel's tty flip buffer may contain leftover bytes from the previous transmission. This happens because:

1. The ESP32 sends its last frame
2. The Python session loop exits (last frame received)
3. A few more bytes from the ESP32 arrive (UART pipelining, protocol overhead) before the ESP32 goes silent
4. These bytes are stored in the kernel buffer
5. The next session starts — `_sync()` sees these stale bytes first

Since PCM audio data is arbitrary binary, any two consecutive bytes can happen to be 0xAA 0x55. The probability is 1/65,536 per pair. In a 32,000-byte-per-second stream, this occurs on average once every $65,536/32,000 \approx 2$ seconds of audio — frequent enough to cause bugs regularly.

The fix:

python

```
def receive_session(self, min_seconds, max_seconds, label="audio"):
    self._ser.reset_input_buffer()    # ← this line is the fix
    ...
```

`serial.Serial.reset_input_buffer()` calls `tcflush(self.fd, TCIFLUSH)`. `tcflush` is a POSIX system call (system call number varies) that instructs the kernel to discard all data currently in the tty input buffer. After this call, the buffer is empty. The next `read()` will block until the ESP32 starts sending fresh data for the new session.

4.3.8 Does It Store All Audio First?

Yes — complete buffering before processing.

The `receive_session()` method accumulates all PCM bytes into a Python `bytearray` called `accum`:

```
python
accum = bytearray()           # empty, initially 0 bytes
while True:
    frame = self._read_frame()
    if frame is None: break    # timeout → session over
    if frame == b'': break     # EOS frame (length=0) → session over
    accum.extend(frame)        # append this frame's bytes to the accumulator
    if len(accum) >= max_bytes: break # reached ENROLL_MAX_S or VERIFY_MAX_S
```

After the loop, `bytes(accum)` is the complete raw PCM session. **No neural network processing happens during audio reception.** The voice worker's thread is entirely consumed by `serial.read()` and bytearray appending during this time.

Only after `receive_session()` returns does the processing pipeline begin (float conversion → normalization → quality check → Mel spectrogram → ONNX inference).

Why not stream in real-time? The ECAPA-TDNN model's Attentive Statistics Pooling (ASP) layer needs all time frames simultaneously to compute its attention-weighted mean and standard deviation. It is mathematically impossible to compute the attention weights for frame 1 until you know frame 200, because the attention weights are functions of all frames together. Streaming inference would require a fundamentally different model (like a causal LSTM or a streaming transformer), which trade accuracy for real-time capability.

Memory cost of buffering: $\text{ENROLL_MAX_S} = 10 \text{ seconds} \times 32,000 \text{ bytes/second} = 320,000 \text{ bytes} \approx 313 \text{ KB}$. $\text{VERIFY_MAX_S} = 6 \text{ seconds} \times 32,000 = 192,000 \text{ bytes} \approx 188 \text{ KB}$. These are tiny compared to the RPi's 8 GB RAM.

4.4 Audio Processing Pipeline — Step by Step

After `receive_session()` returns `raw_pcm_bytes`:

Step 1 — PCM to Float:

python

```
pcm_int16 = np.frombuffer(raw_pcm_bytes, dtype=np.int16)
audio_float = pcm_int16.astype(np.float32) / 32768.0
```

- `np.frombuffer`: zero-copy — wraps the bytes as a numpy array of int16 values
- `/32768.0`: maps the int16 range $[-32768, +32767]$ to float32 range $[-1.0, \sim +1.0]$

For a 5-second enrollment: 80,000 int16 values $\rightarrow$ 80,000 float32 values = 320,000 bytes.

Step 2 — Normalization:

python

```
peak = np.max(np.abs(audio_float))
if peak > 1e-6:
    audio_float = audio_float * (NORM_TARGET_PEAK / peak) # NORM_TARGET_PEAK =
```

This scales the audio so the loudest sample is exactly 0.90 (not 1.0, to avoid clipping in subsequent operations). This removes amplitude variation between speakers who speak at different volumes — the embedding should depend on voice timbre, not loudness.

Step 3 — Quality Check:

Three checks on the normalized audio:

1. `rms = np.sqrt(np.mean(audio_float**2)) > 0.005` — Root Mean Square energy. If $\text{RMS} < 0.005$, the audio is essentially silence (no one spoke). Reject.
2. `zcr = count zero crossings per second > 50` — Zero Crossing Rate. If $\text{ZCR} < 50/\text{second}$, the signal is nearly DC or stuck (UART error, sensor malfunction). Reject.
3. `zcr < 10,000/second` — If $\text{ZCR} > 10,000/\text{second}$, the signal is RF interference or electrical noise (all random sign flips). Reject.

Step 4 — Mel Spectrogram (pure numpy):

This converts the 1D audio waveform into a 2D time-frequency representation.

Why Mel spectrogram? The neural network needs to know "what frequencies are present at what times." A Mel spectrogram captures this, and the Mel frequency scale (logarithmically spaced) matches how the human auditory system perceives pitch — more resolution at low frequencies, less at high frequencies.

Framing:

```
N_FFT      = 400 samples = 25 ms at 16 kHz
HOP_LENGTH = 160 samples = 10 ms at 16 kHz
Overlap     = 240 samples = 15 ms (60% overlap)
Number of frames T = floor((total_samples - N_FFT) / HOP_LENGTH) + 1
For 5 seconds: T = floor((80,000 - 400) / 160) + 1 = 498 frames
```

For each of the T frames:

1. Extract 400 samples starting at $(\text{frame_index} \times 160)$
2. Multiply by Hann window: $(\text{frame} \times \text{np.hanning}(400))$ — tapers the edges to reduce spectral leakage
3. FFT: $(\text{spectrum} = \text{np.fft.rfft}(\text{frame}, n=400)) \rightarrow 201$ complex values
4. Power spectrum: $(\text{power} = |\text{spectrum}|^2 = (\text{spectrum.real}^2 + \text{spectrum.imag}^2)) \rightarrow 201$ float values
5. Mel filterbank: $(\text{mel_energies} = \text{mel_filterbank} @ \text{power}) \rightarrow 80$ float values
 - The filterbank matrix (80×201) is precomputed once at startup
 - Each row is a triangular filter spanning a frequency range on the Mel scale
6. Log: $(\text{log_mel} = \text{np.log}(\text{mel_energies} + 1e-6)) \rightarrow 80$ float values

The $(+1e-6)$ prevents $(\log(0))$ which would produce negative infinity.

Final output: shape $(1, T, 80)$ — 1 batch, T time frames, 80 Mel frequency bins. For 5 seconds: $(1, 498, 80) = 39,840$ float32 values = 159,360 bytes.

Step 5 — ONNX Inference:

python

```
outputs = session.run(None, {input_name: mel_spectrogram})
embedding = outputs[0].squeeze() # remove batch dimension
```

ONNX Runtime runs the ECAPA-TDNN computation graph on the Mel spectrogram. The output is a 1D vector of 192 or 512 floats (depending on the model variant). This is L2-normalized before returning.

Step 6 — Cosine Similarity:

```
python
```

```
def cosine(self, a, b):  
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-12))
```

Compare the live embedding against each stored `.npy` file. The best score is compared against `VERIFY_THRESHOLD = 0.70`.

4.5 ECAPA-TDNN Model Architecture

ECAPA-TDNN (Emphasized Channel Attention, Propagation and Aggregation in Time Delay Neural Networks) is a neural network architecture designed specifically for speaker verification, developed by researchers at KU Leuven and the Idiap Research Institute, and popularized by the SpeechBrain toolkit.

Key architectural components:

1-D convolutions on time: Unlike image CNNs that use 2-D convolutions on (height, width), ECAPA-TDNN uses 1-D convolutions on the time axis of the Mel spectrogram. It treats the 80 Mel frequency bins as "channels" and slides a 1-D kernel along the time axis to capture temporal patterns in speech.

Res2Net blocks: Each major block uses Res2Net — a hierarchical residual connection scheme. The input is split into N smaller feature maps, each processed by a different 1-D convolution, then combined in a hierarchical chain. This creates multi-scale temporal receptive fields within a single layer.

Squeeze-and-Excitation (SE) blocks: After each Res2Net block, channel attention is applied. "Channel" here means Mel frequency bin. The SE block learns to emphasize the frequency channels most informative for distinguishing speakers. For example, the fundamental frequency (F0, ~100-200 Hz for typical speakers) and the first two formants (F1, F2) are highly speaker-specific — SE blocks learn to attend to these.

Attentive Statistics Pooling (ASP): This is the crucial layer that creates a fixed-size output from variable-length input.

Given T time frames, each with D-dimensional features (where D is the hidden dimension of the network):

1. An attention network computes a weight e_t for each time frame t
2. Softmax: $\alpha_t = \exp(e_t) / \sum \exp(e_k)$ — weights sum to 1.0
3. Weighted mean: $\mu = \sum \alpha_t \cdot h_t$ (D-dimensional vector)
4. Weighted standard deviation: $\sigma = \sqrt{\sum \alpha_t \cdot h_t^2 - \mu^2}$ (D-dimensional vector)

5. Concatenate: $[\mu; \sigma] \rightarrow 2\text{D-dimensional vector}$
6. Project to final embedding size (192 or 512-D)

The attention mechanism means the network focuses on the most speaker-discriminative frames — steady vowel sounds — and de-weights noisy frames, silences, and consonants. Whether the input is 2 seconds or 10 seconds, the ASP output is always the same fixed size.

4.6 Voice Enrollment and Verification — Complete Flow

Enrollment:

1. User types name in client $\rightarrow$ client sends `{"command": "register_voice", "name": "Alice"}` over WebSocket
2. Orchestra routes to voice worker: `voice_cmd_q.put({"op": "enroll", "name": "Alice"})`
3. Voice worker calls `receive_session(min_seconds=5.0, max_seconds=10.0)`
 - The TFT display is updated by the voice worker to show "VOICE: SPEAK NOW"
 - The UART listener waits for the ESP32 to start sending audio
4. ESP32 detects the user speaking (button press or VAD), sends frames
5. After 5-10 seconds, EOS frame received or timeout $\rightarrow$ `receive_session()` returns PCM bytes
6. Audio processing pipeline runs (Steps 1-5 above) $\rightarrow$ embedding
7. **Duplicate check:** compare against all `.npy` files in `voiceprint_database_onnx/`:
 - For each existing speaker, load their `.npy` file with `np.load(path)` $\rightarrow$ compute cosine similarity
 - If any score ≥ 0.72 (`DUPLICATE_THRESHOLD`): reject enrollment
 - `DUPLICATE_THRESHOLD` (0.72) is set slightly above `VERIFY_THRESHOLD` (0.70):
 - Two utterances from the SAME person: cosine similarity typically 0.75-0.95
 - Two DIFFERENT people: typically < 0.65
 - The gap between 0.70 and 0.72 prevents false duplicate rejections for people with slightly similar voices
8. If unique: `np.save("voiceprint_database_onnx/alice.npy", embedding)` $\rightarrow$ save to disk
9. Result: `{"status": "success", "message": "alice enrolled"}` $\rightarrow$ back to client

Verification:

1. Client sends `{"command": "auth_voice"}` (or autonomous mode: triggered by ESP32 VAD)
 2. `receive_session(min_seconds=2.0, max_seconds=6.0)` → collect 2-6 seconds
 3. Process → embed → compare against all enrolled `.npy` files
 4. If `best_score >= 0.70`: GRANTED (identity = name of closest match)
 5. Otherwise: DENIED
-

5. Orchestra.py — The Orchestrator

5.1 What `multiprocessing` Is — Deep Explanation

Python's `multiprocessing` module provides an API for creating and managing multiple OS processes. It closely mirrors Python's `threading` module API, but creates true OS processes instead of threads — each with a separate address space, separate file descriptor table, and separate Python interpreter.

How `Process` works at the OS level:

```
python

face_proc = Process(
    target=face_worker_process,
    args=(face_cmd_q, face_res_q, face_busy, shared_logs, shared_users, shared_
    name="face_worker",
    daemon=True)
face_proc.start()
```

`face_proc.start()` calls `os.fork()`:

- `fork()` creates an identical copy of the calling process (copy-on-write)
- In the child process: calls `face_worker_process(face_cmd_q, face_res_q, face_busy, ...)` — the `target` function
- In the parent process: `start()` returns, and `face_proc.pid` contains the child's PID

`daemon=True`: When a non-daemon child process exists and the parent exits, the parent blocks until the child finishes. With `daemon=True`, the child is automatically killed when the parent exits. This prevents zombie processes if the orchestrator crashes.

The `target` function: `face_worker_process` is a regular Python function. When `fork()` creates the child, the child calls this function. The child runs this function forever (it contains the worker's main loop). The child never returns to the parent's `main()` code —

after the fork, parent and child are entirely independent processes sharing only the multiprocessing IPC objects.

5.2 Inter-Process Communication — Detailed Mechanisms

Three IPC primitives are used, all from `multiprocessing`:

`multiprocessing.Queue` — The Command and Result Channels

A `multiprocessing.Queue` is a FIFO queue that can be shared safely between processes. Under the hood:

1. When `Queue()` is created, the OS creates an anonymous **pipe** (two file descriptors: read end and write end)
2. A background thread in the producer process serializes (pickles) each object put into the queue and writes the bytes to the pipe's write end
3. A background thread in the consumer process reads bytes from the pipe's read end and deserializes (unpickles) them back into Python objects

`queue.put({"op": "enroll", "name": "Alice"})` in the orchestrator process:

1. pickle serializes the dict to bytes
2. bytes are written to the pipe (OS pipe buffer, max 64 KB on Linux)
3. If the pipe is full, `put()` blocks until the consumer reads some data

`queue.get_nowait()` in the face worker process:

1. Checks if bytes are available in the pipe (non-blocking check using `select()`)
2. If yes: reads the bytes, unpickles them, returns the dict
3. If no: raises `queue.Empty` exception (which the worker loop catches and ignores)

Important implication: The dict `{"op": "enroll", "name": "Alice"}` is serialized to bytes by pickle, transported through the OS pipe, and deserialized on the other end. This means the face worker receives a **copy** of the dict, not the same object in memory. Any modifications the worker makes to the dict do not affect the orchestrator's copy.

`multiprocessing.Event` — The Busy Flag

A `multiprocessing.Event` is a boolean flag backed by shared memory. `event.set()` sets it to True. `event.clear()` sets it to False. `event.is_set()` reads it.

Under the hood, `multiprocessing.Event` uses `mmap()` to create a shared memory region and a `pthread_mutex_t`/`sem_t` combination for synchronization. Access from multiple

processes is safe.

Usage: The face worker calls `face_busy.set()` when it starts processing a command, and `face_busy.clear()` when it finishes. The orchestrator checks `face_busy.is_set()` before dispatching a new command — if busy, it either waits or rejects the command with "worker is busy."

`multiprocessing.Manager()` — Shared State Across Processes

`Queue` and `Event` are low-level primitives. `Manager()` provides higher-level shared objects — list, dict, etc. — that can be read and written from multiple processes simultaneously.

How Manager works: `multiprocessing.Manager()` starts a **fifth background process** (a manager server process). All other processes interact with shared objects through **proxy objects** — Python objects that, when you call their methods, send the method call over a socket to the manager server, which executes it and returns the result.

```
python

manager = mp.Manager()
shared_logs = manager.list()    # proxy to a list in the manager server process
```

When the face worker calls `shared_logs.append({"event": "GRANTED", ...})`, this:

1. Pickles the method call and arguments
2. Sends them over an OS socket (Unix domain socket) to the manager server
3. Manager server appends the item to the real list it owns
4. Returns success

When the orchestrator calls `list(shared_logs)` to read all log entries, this:

1. Sends a "get slice" request to the manager server
2. Manager server pickles the entire list and sends it back
3. Orchestrator unpickles and returns the list

Why Manager for logs but not for audio data? Manager has overhead per access (socket round-trip, pickle/unpickle). It is suitable for infrequent, small data accesses (appending log events, reading user lists). It would be too slow for per-frame data like camera images.

5.3 The `asyncio` Event Loop in the Orchestrator

The orchestrator's network operations run inside an asyncio event loop. Here is the actual startup:

```
python
```

```
loop = asyncio.get_event_loop()
```

`asyncio.get_event_loop()` creates (or retrieves) the asyncio event loop for the current thread. The event loop is a C structure inside Python's event loop implementation that wraps the Linux `epoll` instance.

```
python
```

```
async with serve(orchestrator.handle_client, "0.0.0.0", 8765, ...):  
    await stop_evt.wait()
```

`serve()` (from the `websockets` library) creates a TCP server socket, binds it to `0.0.0.0:8765`, and registers it with the asyncio event loop's `epoll` instance. Now the event loop monitors this socket for incoming connections.

`await stop_evt.wait()` suspends the coroutine until `stop_evt.set()` is called (by the shutdown signal handler). While waiting, the event loop is free to handle incoming WebSocket connections.

When a client connects:

1. `epoll` signals "new connection ready" on the server socket
2. The event loop calls `accept()` → gets a new socket fd for this client
3. The event loop calls `handle_client(websocket, path)` as a new coroutine
4. Multiple `handle_client` coroutines can run "concurrently" in the event loop — they all share the same thread, switching at `await` points

```
async for raw_msg in websocket:
```

This is an "async for" loop. It is equivalent to:

```
python
```

```
while True:  
    raw_msg = await websocket.recv()    # suspends until a message arrives  
    # process raw_msg
```

Every time `websocket.recv()` suspends (waiting for the next WebSocket frame from the client), the event loop runs other coroutines (e.g., polling the multiprocessing result queue, updating the mDNS advertisement).

5.4 The WebSocket Protocol — What Actually Travels on the Wire

WebSocket is a full-duplex communication protocol built on top of TCP. It starts as an HTTP connection upgrade.

Connection establishment (HTTP Upgrade):

1. Client opens a TCP connection to `[pi-ip]:8765`
2. Client sends an HTTP GET request:

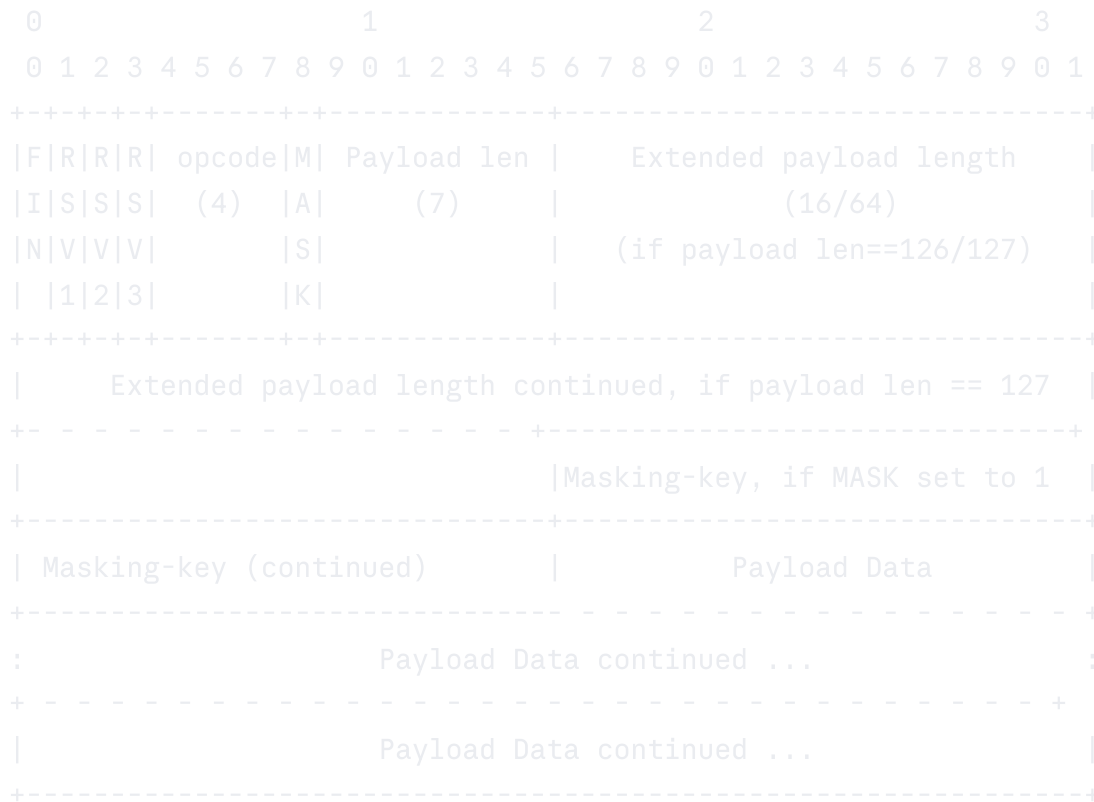
```
GET / HTTP/1.1
Host: 192.168.1.100:8765
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

3. Server responds:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

4. From this point, the TCP connection carries WebSocket frames instead of HTTP

WebSocket Frame Structure (RFC 6455):



For a typical command `{"command": "poweroff"}` (23 bytes):

- FIN=1 (this is the complete message, not fragmented)
- Opcode=0x1 (text frame — UTF-8 content)
- MASK=1 (client-to-server frames must be masked per RFC 6455)
- Payload length = 23
- Masking key = 4 random bytes (e.g., `0xDE 0xAD 0xBE 0xEF`)
- Payload = XOR of each byte with masking key cycling through 4 bytes

Server-to-client frames do not use masking (MASK=0).

The `websockets` Python library handles all of this framing automatically. Python code just calls `await websocket.send("string")` or `raw_msg = await websocket.recv()`.

Ping/Pong frames: With `ping_interval=30`, the `websockets` library sends a WebSocket PING frame (opcode 0x9) every 30 seconds. The client must respond with a PONG frame (opcode 0xA) within `ping_timeout=120` seconds. If no PONG arrives in 120 seconds, the server closes the connection. This detects network failures where the TCP connection silently dies (cable unplugged, Wi-Fi disconnected) without a TCP FIN being sent.

5.5 The `_route()` Function — Command Dispatch

Every WebSocket message goes through `_route()`. This is a standard if-elif dispatch:

python

```
async def _route(self, command, name, data, websocket):
    if command == "auth_face":
        return await self._dispatch("face", {"op": "verify"}, websocket)
    elif command == "register_face":
        return await self._dispatch("face", {"op": "enroll", "name": name}, websocket)
    elif command == "delete":
        face_r = await self._dispatch("face", {"op": "delete", "name": name})
        voice_r = await self._dispatch("voice", {"op": "delete", "name": name})
        return {"status": "success", "face": face_r, "voice": voice_r}
    elif command == "reboot":
        ...
```

`_dispatch(worker, cmd)` is an async coroutine that:

1. Puts `cmd` on the worker's command queue
2. Suspends (`await asyncio.sleep(0.05)`) in a polling loop, waiting for a result on the result queue
3. Returns the result dict when it arrives (or times out after 120 seconds)

Why `await asyncio.sleep(0.05)` instead of blocking?

`asyncio.sleep(0.05)` suspends the current coroutine for 50 ms and releases control to the event loop. During those 50 ms, the event loop can handle other WebSocket messages, send ping frames, etc. This is the cooperative multitasking at work — the dispatch function politely yields the event loop during the wait.

If it were `time.sleep(0.05)` instead (blocking sleep), the event loop would be frozen for 50 ms every poll iteration. The WebSocket server would become unresponsive for seconds while waiting for the worker to finish.

5.6 mDNS/ZeroConf Advertisement — Protocol Details

What mDNS is: mDNS (Multicast DNS, RFC 6762) extends the DNS protocol to work without a DNS server, using IP multicast. Instead of sending queries to a configured DNS server, an mDNS client sends DNS query packets to the multicast address `224.0.0.251` on UDP port 5353. All devices on the local network receive these packets (because

(`224.0.0.251`) is a link-local multicast address). Any device that knows the answer responds with a DNS reply — also sent to the multicast address (so all devices can cache it).

What DNS-SD is: DNS-SD (DNS-based Service Discovery, RFC 6763) defines a convention for advertising services over mDNS. Services are registered as DNS SRV and TXT records with a specific naming scheme: `{service-name}.{service-type}.local.`

What the `zeroconf` Python library does:

```
python

zc = Zeroconf(ip_version=IPVersion.V4Only)
```

Creates a UDP socket, binds it to port 5353, joins the multicast group (`224.0.0.251`) using `setsockopt(IP_ADD_MEMBERSHIP)`. On Linux, this tells the network interface to accept packets addressed to (`224.0.0.251`).

```
python

info = ServiceInfo(
    "_biometric-auth._tcp.local.",          # service type (user-defined)
    "BiometricServer._biometric-auth._tcp.local.", # service instance name
    addresses=[socket.inet_aton("192.168.1.100")], # IP as 4 bytes
    port=8765,
    properties={"version": "3.0.0", "workers": "face+voice"},
)
await zc.async_register_service(info)
```

`async_register_service()` sends mDNS announcement packets (DNS PTR, SRV, A, TXT records) to (`224.0.0.251:5353`). The packet contains:

- PTR record: `_biometric-auth._tcp.local. → BiometricServer._biometric-auth._tcp.local.`
- SRV record: `BiometricServer._biometric-auth._tcp.local. port=8765 target=raspberrypi.local.`
- A record: `raspberrypi.local. → 192.168.1.100`
- TXT record: `version=3.0.0 workers=face+voice`

Any device on the LAN browsing for `_biometric-auth._tcp.local.` services sees these records and knows the server is at `ws://192.168.1.100:8765`.

The `zeroconf` library also handles **probing** (checking that no other device has the same service name before registering) and **conflict resolution** (renaming if a conflict is detected).

5.7 Network Supervisor Coroutine

python

```
async def run_network_supervisor(orchestrator, port):
    while True:
        ip = await wait_for_lan_ip(LAN_CHECK_INTERVAL_S)
        stop_evt = asyncio.Event()
        t1 = asyncio.create_task(run_server(orchestrator, ip, port, stop_evt))
        t2 = asyncio.create_task(run_zeroconf(ip, port, stop_evt))
        while True:
            await asyncio.sleep(LAN_CHECK_INTERVAL_S)
            if get_local_ip() != ip:
                log.warning("[NET] IP changed - restarting")
                break
        stop_evt.set()
        await asyncio.gather(t1, t2, return_exceptions=True)
```

`asyncio.create_task()` schedules a coroutine to run in the event loop concurrently with the current coroutine. Both the WebSocket server and the ZeroConf advertisement run as separate "tasks" in the same event loop thread.

`asyncio.gather(t1, t2)` waits for both tasks to finish (after `stop_evt.set()` signals them to stop). `return_exceptions=True` means if either task raises an exception, `gather` returns the exception object instead of propagating it — so one failing task does not prevent the other from shutting down cleanly.

`wait_for_lan_ip()` polls `get_local_ip()` every second until a non-loopback IPv4 address is found (e.g., `192.168.1.100`). This handles startup when the network interface is not yet up.

5.8 Reboot and Power Off — Complete Mechanism

When the client clicks "Reboot Pi":

On the RPi, in `_route()`:

python

```

elif command == "reboot":
    log.info("[WS] Reboot requested by client")
    _log_event(self.shared_logs, "server", "REBOOT_REQUESTED")
    self.face_cmd_q.put({"op": "stop"})
    self.voice_cmd_q.put({"op": "stop"})
    import subprocess as _sp
    asyncio.get_event_loop().call_later(
        2.0, lambda: _sp.Popen(["sudo", "reboot"]))
    return {"status": "success", "message": "Rebooting..."}

```

Step-by-step:

1. `log.info(...)` — Python's `logging` module writes to the log file and/or console
2. `_log_event(...)` — appends a dict to the Manager-shared log list
3. `face_cmd_q.put({"op": "stop"})` — serializes `{"op": "stop"}`, writes to the face command pipe. The face worker will pick this up on its next queue check, run its cleanup code, and exit.
4. `voice_cmd_q.put({"op": "stop"})` — same for voice worker
5. `asyncio.get_event_loop().call_later(2.0, lambda: _sp.Popen(["sudo", "reboot"]))` — this is crucial:
 - `call_later(2.0, fn)` schedules `fn` to be called by the event loop after 2 seconds
 - It does NOT block — it registers a timed callback and returns immediately
 - The function then returns `{"status": "success", "message": "Rebooting..."}`
 - This return value is sent back to the client over WebSocket (this takes < 1 ms)
 - **2 seconds later**, after the WebSocket response has definitely reached the client, the event loop fires the callback
6. `_sp.Popen(["sudo", "reboot"])`:
 - `Popen` is Python's `subprocess.Popen` class constructor
 - Unlike `subprocess.run()`, `Popen` does NOT wait for the child to finish — it just spawns the child and returns immediately
 - `Popen(["sudo", "reboot"])` calls `fork()` to create a child process
 - The child calls `execve("/usr/bin/sudo", ["sudo", "reboot"], environ)`
 - `sudo` looks up the user in `/etc/sudoers`. On Raspberry Pi OS, the `pi` user has the line `pi ALL=(ALL) NOPASSWD: ALL`, which means sudo runs the command as root without asking for a password
 - `sudo` calls `execve("/sbin/reboot")` to become the `reboot` program
 - `reboot` calls the Linux `reboot(LINUX_REBOOT_CMD_RESTART)` system call

- The kernel sends `SIGTERM` to all processes (`PID > 1`), waits ~5 seconds, sends `SIGKILL`
- The kernel unmounts all filesystems (calling `fsync()` to flush pending writes)
- The kernel writes the halt code to the BCM2711 SoC power management registers
- The SoC resets the ARM cores

Why `Popen` instead of `run()`? If the code called `subprocess.run(["sudo", "reboot"])`, it would wait for `sudo reboot` to finish. But `sudo reboot` never finishes (the process is killed by the kernel as part of shutdown). The asyncio event loop would be blocked forever. Using `Popen` (non-blocking spawn) avoids this — the child runs independently, and the orchestrator continues running (long enough for the kernel to kill it).

Why the 2-second delay? Without it:

- `_route()` returns `{"status": "success", "message": "Rebooting..."}`
- `handle_client()` calls `await websocket.send(json.dumps(response))` — this sends the response
- Before the TCP ACK from the client arrives, `sudo reboot` kills the network stack
- The client never receives the response and shows "Error" instead of "Rebooting..."

With 2 seconds, there is ample time for the TCP data to be ACK'd by the client.

6. Client.py — The Remote TUI

6.1 What `Textual` Is

TUI stands for Text User Interface — a graphical-looking interface that runs entirely inside a terminal (command prompt window), using ANSI escape codes to position text, draw boxes, change colors, etc. Think of `ncurses` programs like `htop` or `vim`, but modern and Python-native.

`Textual` is a Python framework for building TUIs. It is built on top of `Rich` (another Python library for colored terminal output) and provides:

- **Widgets:** `Button`, `Input`, `DataTable`, `RichLog`, `Select`, `Switch`, `Label`, `Static`, `TabbedContent`, `TabPane` — these are objects that display interactive UI elements
- **CSS-like styling:** Layout and appearance are controlled by a CSS-like string inside the app class. `Textual` handles terminal-compatible rendering.
- **Event system:** Clicking a button calls `on_button_pressed()`, typing in an input calls `on_input_changed()`, etc.

- **Screen management:** `push_screen(ConfirmScreen(...))` overlays a modal dialog on top of the current screen. `pop_screen()` removes it.
- **Async integration:** Textual's internal event loop is built on asyncio. When you click a button, the `on_button_pressed` method runs as a coroutine in the Textual event loop. This is why `await self._send(...)` works inside button handlers.

`run_worker(coroutine, name="...")`: Textual's way of running an async function in a background task that does not block the UI. The UI remains responsive while the worker runs. Workers are like asyncio tasks managed by Textual.

6.2 What `asyncio.Lock` Is

```
python
```

```
self._cmd_lock = asyncio.Lock()
```

`asyncio.Lock` is the asyncio equivalent of a mutex. It prevents two coroutines from running a critical section simultaneously.

```
python
```

```
async with self._cmd_lock:
    res = await self._send({"command": "auth_face"})
```

`async with` is Python's asynchronous context manager. When a coroutine reaches `async with self._cmd_lock:`, it tries to acquire the lock. If the lock is free: acquires it immediately and continues. If the lock is held by another coroutine: suspends (releases the event loop) until the lock is released.

Why it is needed:

The client has two coroutines running concurrently:

1. A **log polling coroutine** that runs every 3 seconds, sending `{"command": "get_logs"}` and reading the response
2. **Button-triggered coroutines** that send commands like `{"command": "auth_face"}` and wait for responses

If both run simultaneously and both call `await self.ws.send(...)` and `await self.ws.recv()` at the same time, the responses would interleave. The log poller might read the face auth result, and the face auth coroutine might read the log response — both getting the wrong data.

The `_cmd_lock` serializes all WebSocket traffic. Only one coroutine can be inside the `async with self._cmd_lock:` block at a time.

Difference from `threading.Lock`: With `threading.Lock`, `acquire()` blocks the OS thread. With `asyncio.Lock`, `async with lock:` suspends only the coroutine — the event loop continues running other coroutines. This is the cooperative nature of asyncio.

6.3 mDNS Discovery — Client Side

```
python
```

```
self._zc = Zeroconf()
self._listener = _MDNSListener(self._on_server_found)
self._browser = ServiceBrowser(self._zc, "_biometric-tcp.local.", self._l
```

`Zeroconf()`: Opens a UDP socket on port 5353 (in a background thread) and joins the multicast group `224.0.0.251`. This enables the socket to receive mDNS packets.

`ServiceBrowser`: Sends an mDNS query (DNS PTR record query for `_biometric-auth._tcp.local.`) to `224.0.0.251:5353`. This query asks: "Is anyone offering `_biometric-auth._tcp.local.` services?" The query is also periodic — sent repeatedly on a schedule to handle new servers coming online.

When the RPi's mDNS announcement arrives:

The `Zeroconf` background thread receives the UDP multicast packet on the socket. It parses the DNS records (PTR, SRV, A, TXT). The `ServiceBrowser` calls `listener.add_service(zc, type, name)`.

`_MDNSListener.add_service()`:

```
python
```

```
def add_service(self, zc, type_, name) -> None:
    url = self._url_from_info(zc, type_, name) # resolve to ws://ip:port
    if url and url not in self._seen:
        self._seen.add(url)
        self._cb(url) # calls self._on_server_found(url)
```

`_url_from_info()` calls `zc.get_service_info(type_, name)` to look up the SRV and A records for this service, extracting the IP address (as 4 raw bytes) and port number, then constructs `ws://192.168.1.100:8765`.

The `_seen` set prevents reconnecting to a server the client already knows about.

`update_service()` removes the URL from `_seen` when the server is updated (restarted with the same name), allowing reconnection.

`_on_server_found(url)`: Stores the URL and triggers a WebSocket connection attempt. If a manual URL is configured in Settings, mDNS discovery is skipped.

6.4 WebSocket Connection and Reconnection

python

```
async def _connect_loop(self):
    while True:
        url = self.server_url or await self._wait_for_mdns()
        try:
            async with websockets.connect(url, ...) as ws:
                self.ws = ws
                self._on_connected()
                async for msg in ws:          # receive loop
                    self._on_message(msg)
        except Exception:
            self._on_disconnected()
            await asyncio.sleep(backoff_time) # exponential backoff
```

`websockets.connect(url)` is an async context manager that:

1. Resolves the hostname (or uses the IP directly)
2. Opens a TCP connection (`socket.connect()`)
3. Performs the WebSocket HTTP Upgrade handshake
4. Returns a `WebSocketClientProtocol` object with `send()` and `recv()` methods

`async for msg in ws:` receives messages continuously until the connection closes. Each received message is a Python string (JSON text).

Exponential backoff: When the connection fails (RPI offline, Wi-Fi issue), the client waits before retrying: 2s → 5s → 10s → 20s → 30s → 60s → 60s → ... Each reconnect attempt uses the next backoff time, up to a maximum of 60 seconds.

6.5 Threshold Management — How It Works End-to-End

Fetching on connect:

python

```
async def _fetch_thresholds_locked(self):
    res = await self._send({"command": "get_thresholds"}, timeout=10.0)
    if res and res.get("status") == "success":
        for k in ("face_similarity", "face_min_score_gap",
                 "face_enroll_duplicate", "face_min_gallery",
                 "voice_verify", "voice_duplicate"):
            if k in res:
                self._settings[k] = res[k]
        _save_settings(self._settings)
```

After connecting, the client sends `{"command": "get_thresholds"}`. Orchestra dispatches to both workers. Each worker's `get_thresholds` handler returns its current module-level constants. Orchestra merges the results into one dict. The client stores them in `self._settings` and saves to `~/.biometric_client_settings.json`.

Pushing after settings change:

python

```
async def _push_thresholds(self):
    async with self._cmd_lock:
        payload = {
            "command": "set_thresholds",
            "face_similarity": self._settings.get("face_similarity", 0.75),
            ...
        }
    res = await self._send(payload, timeout=10.0)
```

Orchestra's `set_thresholds` handler splits the keys by worker and dispatches to face and voice workers. Each worker's handler updates the Python **module object's** global variables:

python

```
# In face worker:
elif op == "set_thresholds":
    if "face_similarity" in cmd:
        _main_mod.SIMILARITY_THRESHOLD = float(cmd["face_similarity"])
```

`_main_mod` is the `main` module object (imported with `import main as _main_mod`). Setting `_main_mod.SIMILARITY_THRESHOLD = 0.78` changes the variable in the `main` module's global namespace. The next time `match_probe()` reads `SIMILARITY_THRESHOLD`, it reads the new value — because Python module globals are looked up at call time, not at import time.

6.6 Selective Delete — How It Works

The client sidebar contains:

```
python
yield Select(
    options=[
        ("Face + Voice", "both"),
        ("Face Only", "face"),
        ("Voice Only", "voice"),
    ],
    value="both",
    id="del-mode",
)
```

When delete is confirmed, the button handler reads the selection and sends:

- Mode "both": `{"command": "delete", "name": "Bob"}`
- Mode "face": `{"command": "delete_face", "name": "Bob"}`
- Mode "voice": `{"command": "delete_voice", "name": "Bob"}`

Orchestra's `_route()` handles each differently:

- `delete`: dispatches to both face AND voice worker's `delete` op
- `delete_face`: dispatches only to face worker's `delete` op; sets voice result to `{"status": "skipped"}`
- `delete_voice`: dispatches only to voice worker's `delete` op; sets face result to `{"status": "skipped"}`

The face worker's delete op removes the entry from the pickle dictionary and saves. The voice worker's delete op removes the `.npy` file from `voiceprint_database_onnx/`.

7. End-to-End Command Flow Examples

7.1 Pressing "Power Off" on the Client — Complete Trace

This is the most important example to understand for your examination. Every step is traced through every layer.

Step 1 — User clicks "🔌 Power Off Pi" button

Step 2 — `on_button_pressed()` fires:

python

```
if btn == "btn-poweroff":
    def _do_poweroff():
        self.run_worker(self._power_command("poweroff"), name="poweroff-pi")
    self.push_screen(ConfirmScreen(
        "▲ Confirm Power Off",
        "Shut down the Raspberry Pi?\nYou will need physical access to turn it
        _do_poweroff,
    ))
```

`push_screen(ConfirmScreen(...))` overlays a modal dialog. The `ConfirmScreen` displays two buttons: "✓ Yes" and "X No". The `_do_poweroff` function is captured in a Python **closure** — it is stored as a callable associated with the "Yes" button.

Step 3 — User clicks "✓ Yes"

`ConfirmScreen.on_button_pressed()` fires. It calls `self._on_confirm()` which calls `_do_poweroff()`. The screen pops itself with `self.app.pop_screen()`.

Step 4 — `_do_poweroff()` starts a Textual worker

python

```
self.run_worker(self._power_command("poweroff"), name="poweroff-pi")
```

`self._power_command("poweroff")` creates a coroutine object — it does NOT start executing yet. `run_worker()` schedules it as an asyncio task in Textual's event loop.

Step 5 — `_power_command("poweroff")` executes:

python

```

async def _power_command(self, cmd: str) -> None:
    label = "Power Off"
    async with self._cmd_lock:      # acquire lock (suspends if log poller hold
        self._action("Power Off - sending...")
        self._sys_log("[yellow]► Power Off requested...[/yellow]")
        res = await self._send({"command": "poweroff"}, timeout=10.0)

```

`self._send()` serializes the dict to JSON and calls `await self.ws.send(json_string)`.

Step 6 — What `self.ws.send()` does at the TCP level:

The `websockets` library takes the string `'{"command": "poweroff"}'` (23 characters = 23 UTF-8 bytes), wraps it in a WebSocket text frame:

```

Frame bytes (from client to server):
0x81          = FIN=1, RSV1-3=0, opcode=0x1 (text)
0x97          = MASK=1, payload_length=23 (0x17 = 23, plus bit 7 set for
mask)
0xDE 0xAD 0xBE 0xEF = masking key (4 random bytes)
masked payload:  each byte of '{"command": "poweroff"}' XOR'd with cycling
masking key

```

This is 2 (header) + 4 (masking key) + 23 (payload) = **29 bytes** sent over TCP.

The TCP socket `send()` call writes these 29 bytes to the kernel's TCP send buffer. The TCP stack encapsulates them in a TCP segment with appropriate sequence numbers, adds an IP header with source and destination IP addresses, and sends it to the network interface (Wi-Fi or Ethernet).

Step 7 — Response arrives at the client:

The server sends back `{"status": "success", "message": "Powering off\u2026"}` (the `...` ellipsis is the Unicode character U+2026, encoded as `\u2026` in JSON, which is 3 bytes in UTF-8). The client's `await self.ws.recv()` returns this string. The client shows a notification toast.

Step 8 — On the RPi, `_route()` processes the command:

```
python
```

```

elif command == "poweroff":
    log.info("[WS] Power off requested by client")
    _log_event(self.shared_logs, "server", "POWEROFF_REQUESTED")
    self.face_cmd_q.put({"op": "stop"})
    self.voice_cmd_q.put({"op": "stop"})
    import subprocess as _sp
    asyncio.get_event_loop().call_later(
        2.0, lambda: _sp.Popen(["sudo", "poweroff"]))
    return {"status": "success", "message": "Powering off..."}

```

`self.face_cmd_q.put({"op": "stop"})` — pickles the dict, writes to OS pipe. The face worker will receive this on its next queue poll, stop its camera loop, close the display, and exit its `face_worker_process()` function. The OS will clean up the child process.

`asyncio.get_event_loop().call_later(2.0, lambda: _sp.Popen(["sudo", "poweroff"]))` — registers a 2-second timer. Returns immediately. The function then returns the response dict, which is JSON-encoded and sent back to the client.

Step 9 — 2 seconds pass:

The asyncio event loop's internal timer fires. The registered lambda function is called:

`_sp.Popen(["sudo", "poweroff"])`.

`Popen.__init__()` calls `os.fork()`:

- **Child process:** calls `os.execvpe("sudo", ["sudo", "poweroff"], env)` — the child's memory image is replaced by the `sudo` binary
- **Parent process (orchestrator):** `Popen.__init__()` returns immediately

In the child (now `sudo`):

- `sudo` checks `/etc/sudoers` → `pi ALL=(ALL) NOPASSWD: ALL` → allowed, no password needed
- `sudo` calls `execvpe("poweroff", ["poweroff"], env)` — child's memory replaced by `poweroff`

In the `poweroff` binary:

- Calls `reboot(LINUX_REBOOT_CMD_POWER_OFF)` system call
- The kernel takes over: initiates shutdown sequence

Step 10 — Linux kernel shutdown sequence:

1. Kernel sends SIGTERM to all processes (PID > 1) — face_worker, voice_worker, orchestrator, etc.
2. Waits up to 5 seconds for processes to exit cleanly
3. Sends SIGKILL to any remaining processes (cannot be caught or ignored)
4. Calls filesystem sync (`fsync()`) on all mounted filesystems — writes any buffered data to the SD card
5. Unmounts all filesystems
6. Calls hardware power-off via BCM2711 SoC power management registers → 3.3V rail de-energized → RPi powers off

Step 11 — Client detects disconnect:

The TCP connection to the RPi is severed when the network stack shuts down. On the client, `await self.ws.recv()` (in the receive loop) raises a `ConnectionClosedError` or `ConnectionResetError`. The `except Exception` block in `_connect_loop` catches it, calls `_on_disconnected()`, which sets the status label to "OFFLINE" in red, and starts the reconnect backoff timer.

7.2 Pressing "Verify Face" on the Client

1. **Client:** `on_button_pressed("btn-auth-face")` → `_command_worker("btn-auth-face")` → `_send({"command": "auth_face"})`
 - JSON serialized, WebSocket frame sent over TCP
2. **Orchestra** `_route()`: matches `command == "auth_face"` → calls `await self._dispatch("face", {"op": "verify"}, websocket)`
3. `_dispatch("face", ...)`:
 - `face_cmd_q.put({"op": "verify"})` — puts command in the pipe
 - Enters polling loop: `await asyncio.sleep(0.05)` → check `face_res_q.get_nowait()`
 - Polls every 50 ms, up to 120 seconds
4. **Face worker main loop (on Core 1):**
 - `cmd = face_cmd_q.get_nowait()` → gets `{"op": "verify"}`
 - `face_busy.set()` — marks worker as busy
 - `_handle_command({"op": "verify"})` is called
5. **Inside** `_handle_command({"op": "verify"})`:
 - `capture_probe()` is called:
 - TFT: "LOOK AT CAMERA - 3s"

- Countdown loop: preview frame → MediaPipe → check conditions every 100 ms
- After 3 consecutive seconds with valid face: countdown complete
- `camera._to_hires()` → stop preview → reconfigure → restart hi-res
- `camera.capture_array()` → 3280×2464 frame in RAM (23 MB numpy array)
- MediaPipe on hi-res frame → eye keypoints
- `align_face()` → 112×112 crop
- `cv2.imwrite("temp_probe.jpg", aligned_crop_bgr)` → JPEG to disk
- `get_face_embedding("temp_probe.jpg")` → fork+exec Rust binary → 128-D numpy array
- `camera._to_preview()` → back to 30 FPS preview
- `match_probe(embedding, db)` is called:
 - Layer 1: `len(db) >= 2`? Yes → continue
 - For each enrolled person: `score = np.dot(embedding, db[name])`
 - Sort by score → best, second_best
 - Layer 2: `best_score >= 0.75`? (say 0.87 for Alice) → continue
 - Layer 3: `best_score - second_score >= 0.10`? (say 0.87 - 0.71 = 0.16) → continue
 - Result: `{"granted": True, "user": "Alice", "score": 0.87}`
- TFT: "✓ GRANTED – Alice" (green) for 2 seconds
- `_log_event(shared_logs, "face", "GRANTED", user="Alice", confidence=0.87, ...)`

6. Face worker puts result:

- `face_res_q.put({"status": "success", "granted": True, "user": "Alice", "score": 0.87})`
- `face_busy.clear()`

7. Orchestra `_dispatch()`:

- Polling loop: `face_res_q.get_nowait()` → returns the result dict
- Returns `{"status": "success", "granted": True, "user": "Alice", "score": 0.87}` to `_route()`

8. Orchestra `handle_client()`:

- `await websocket.send(json.dumps(result))` → JSON string sent over TCP to client

9. Client:

- `await self.ws.recv()` returns the JSON string

- `json.loads()` → dict
 - `_show_auth_result()` displays: green "✓ GRANTED — Alice (0.87)" in the Face Log tab
-

8. Suggested Improvements for Commercial Grade

8.1 Security

Encrypted WebSocket (WSS):

Currently using plain `ws://` — all JSON traffic (user names, biometric scores, threshold values, reboot commands) is transmitted as unencrypted TCP data. Anyone on the same Wi-Fi network can capture these packets with Wireshark and see everything.

WSS (WebSocket Secure) is WebSocket over TLS, exactly like HTTPS. The `websockets` library supports WSS natively. Requires: generating a TLS certificate (self-signed or from Let's Encrypt), configuring the server with `ssl_context`, and the client with `ssl=ssl_context`. After TLS handshake, all WebSocket data is AES-encrypted inside the TLS session.

Authentication on WebSocket:

Currently, any device on the LAN that connects to `ws://pi-ip:8765` can send any command — including `{"command": "poweroff"}` or `{"command": "delete", "name": "Alice"}`. There is no authentication.

A simple fix: after WebSocket connection, require the client to send a token: `{"auth": "secret_password_123"}`. Server checks it. If wrong, closes the connection. A proper fix: challenge-response authentication using HMAC (Hash-based Message Authentication Code).

Liveness Detection (Face):

The current system accepts a printed photograph of Alice held in front of the camera. MediaPipe detects the printed face as a real face — it has no way to tell the difference. The Rust binary embeds the printed face and it matches Alice's enrollment embedding.

Anti-spoofing approaches:

- **Blink challenge:** Ask the user to blink. Detect eye closure using MediaPipe eye keypoints. A photo cannot blink.
- **Depth (IR camera):** Add an infrared depth camera (like RealSense D415). A flat photo has no depth variation. A real face has 5–10 cm depth variation between nose tip and ears.

- **Texture analysis:** A printed photo has regular printing artifacts (halftone pattern) visible at the right scale. A moiré interference pattern (from photographing a screen) is detectable.

Liveness Detection (Voice):

A recording of Alice's voice played through a loudspeaker can spoof the voice biometric. Countermeasures:

- **Text-dependent verification:** Instead of free speech, play a random challenge phrase (e.g., "Say: green elephant seven") and verify that the user says those exact words (using speech recognition in addition to speaker verification).
- **Channel mismatch detection:** A loudspeaker + room adds characteristic acoustic reverberation. Models trained on clean speech will show slightly lower scores for replayed audio.

Encrypted Database:

`faces_db.pkl` and `.npy` files are stored as plaintext on the SD card. If the SD card is removed, all biometric templates are exposed — these cannot be revoked once compromised (you cannot change your face). Encrypt using AES-256 with a key derived from a PIN (PBKDF2 or Argon2 key derivation function) or stored in a hardware security module (TPM).

8.2 Reliability

Watchdog Process:

If the face or voice worker crashes (numpy exception, camera disconnect, UART error), the orchestrator continues running but that worker's commands will time out silently. The client will see "Worker not responding." Add a supervisor that calls `face_proc.is_alive()` every 5 seconds and calls `face_proc.start()` again if the worker has died.

Atomic Database Writes:

`pickle.dump(db, open("faces_db.pkl", "wb"))` opens the file and writes to it. If power is lost during the write (the SD card write cycle is typically 1-4 ms), the file is partially written and corrupted — all enrolled users lost.

The fix:

```
python
```

```
import tempfile, os
with tempfile.NamedTemporaryFile(delete=False, suffix=".tmp", dir=".") as f:
    pickle.dump(db, f)
    temp_path = f.name
os.rename(temp_path, "faces_db.pkl") # rename is atomic on Linux ext4
```

`os.rename()` is guaranteed by POSIX to be atomic — either the old file exists or the new file exists, never a partial state. This eliminates database corruption on power loss.

Persistent Threshold Storage:

Currently, threshold changes (from the client's Settings screen) update the workers' module-level variables in RAM. When the RPi reboots, these variables reset to the hardcoded defaults in `main.py` and `rpi4_ecapa_voice_biometric_v2.py`. The settings on the client (stored in `~/.biometric_client_settings.json`) remember the last values, and the client pushes them again on reconnect. But if the client is offline and the RPi reboots, the thresholds reset.

Fix: when the orchestra receives `set_thresholds`, also write the new values to a JSON config file on disk (e.g., `/home/pi/.biometric_thresholds.json`). At worker startup, read this file before falling back to the hardcoded defaults.

UPS and Graceful Power Loss:

The Linux ext4 filesystem has a journal, but a sudden power cut can still corrupt the journal or leave orphaned files. A UPS (Uninterruptible Power Supply) hat like PiJuice monitors battery voltage and, when power is lost, signals the RPi via GPIO to trigger a clean shutdown. The RPi calls `sudo poweroff` programmatically before the battery is exhausted.

8.3 Performance

Keep the Rust Binary Loaded (Persistent Process):

The largest performance waste in the face subsystem is restarting the Rust binary for every face embedding. Each restart costs: process fork (~2 ms) + ELF loading and dynamic linking (~5 ms) + MobileFaceNet ONNX model loading into tract (~200–500 ms) = **~400 ms overhead before inference begins**.

The solution: run the Rust binary as a long-lived server process that:

1. Loads the model once at startup
2. Reads image paths from stdin (one per line) or a Unix domain socket
3. Writes embedding JSON to stdout
4. Never exits

Python uses `subprocess.Popen()` (non-blocking) to start the Rust server once, keeps the `Popen` object, and communicates by writing to `stdin` and reading from `stdout`. This reduces per-embedding overhead from ~400 ms to ~1 ms (just the pipe I/O).

Camera Zero-Copy:

Currently the pipeline is: Camera DMA → RAM (24 MB numpy array) → CPU color conversion → CPU affine warp → CPU JPEG encode → disk write → Rust: disk read → CPU JPEG decode → CPU resize → CPU inference.

Zero-copy optimization: pass the aligned face crop as raw pixel bytes through a Unix socket or pipe to the Rust binary, skipping the JPEG encode/decode round-trip. Saves ~10–20 ms of encode+decode time per face.

Voice Feature Extraction on ESP32:

The ESP32 (with 240 MHz dual-core Xtensa LX7 and 520 KB SRAM) is capable of computing the Mel spectrogram. If the ESP32 sends (T, 80) float features over UART instead of raw PCM, the UART bandwidth drops from 32,000 bytes/second (raw PCM) to approximately 8,000 bytes/second (Mel features at 10 ms per frame × 80 floats × 4 bytes = 3,200 bytes/second), and the RPi avoids the numpy FFT computation entirely.

8.4 Scalability

Multi-Sample Enrollment:

A single embedding from one capture session is susceptible to variation: lighting changes, glasses on/off, beard growth, aging. Enrolling 3–5 samples (from different sessions, different lighting conditions) and storing their average provides a more robust template.

Averaging embeddings: `template = (emb1 + emb2 + emb3) / 3` followed by L2 normalization. The averaged template represents the "center of mass" of the person's embedding distribution, reducing false rejections.

Database Indexing for Many Users:

Currently `match_probe()` computes cosine similarity against every enrolled person — this is $O(N)$ per verification. For $N=50$ users, this is 50 dot products of 128-D vectors = 50×256 floating-point operations ≈ negligible on RPi4. But for $N=1000$ users: still < 1 ms. For $N=1,000,000$ (large facility): need approximate nearest neighbor search (FAISS from Facebook AI Research, or Annoy from Spotify). These build an index that finds approximate nearest neighbors in $O(\log N)$ instead of $O(N)$.

8.5 User Experience

Multi-Factor Score Fusion:

Currently, face and voice are independent. A more robust system computes a combined score: $\text{final} = 0.6 \times \text{face_score} + 0.4 \times \text{voice_score}$. The weights can be tuned to reflect the relative reliability of each modality. If one modality fails (user has a cold affecting voice, or covers their face), the fusion score is degraded but the other modality can compensate.

Audit Log Integrity:

The CSV log can be modified by anyone with SSH access to the RPi. For a forensic audit trail, use hash chaining: each log entry includes a SHA-256 hash of (previous_hash + current_entry_data). Any tampering with a historical entry invalidates all subsequent hashes. This is the same principle used in blockchain, but applied to a simple append-only file.

OTA Updates:

Currently, updating the system requires SSHing into the RPi and manually pulling from Git. An OTA (Over The Air) update mechanism would have the RPi periodically poll a server for new versions, download them, verify their cryptographic signature (to prevent tampered updates), and apply them. The system would then restart with the new code.

Web Dashboard:

The Textual TUI runs only in a terminal. A web-based dashboard using FastAPI (Python web framework) and a JavaScript frontend would be accessible from any browser — phone, tablet, or PC — without installing any software. Real-time event streaming would use Server-Sent Events (SSE) or a browser WebSocket connection to the same Orchestra server.

End of Document — Version 2.0

All original content from Version 1.0 is preserved and expanded. No information was removed. Every Python concept, library, and mechanism is explained from first principles for a computer engineering student who understands operating system fundamentals.